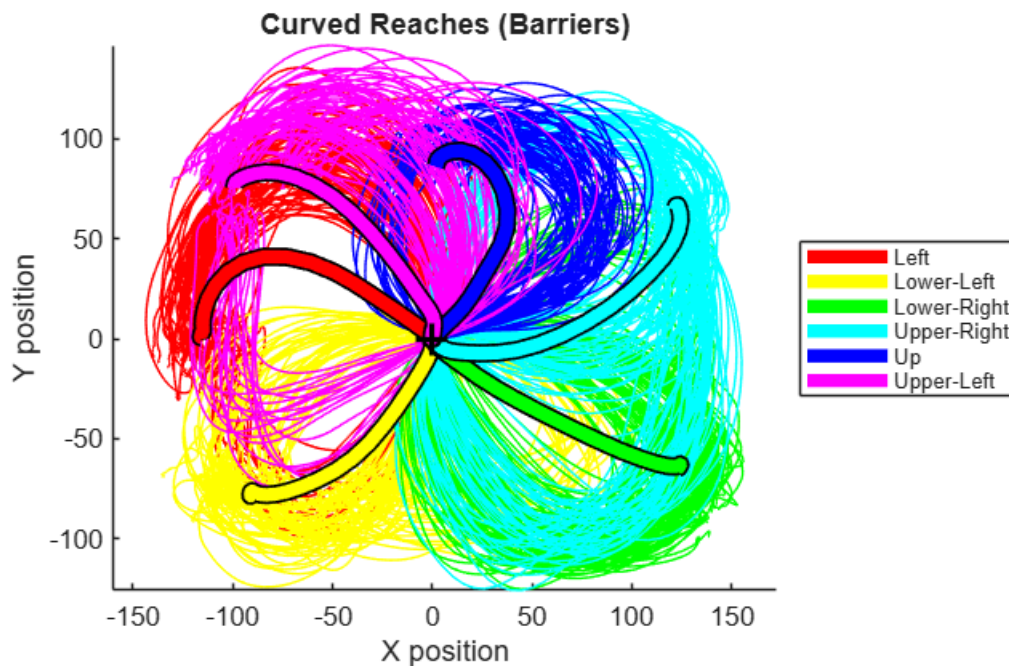
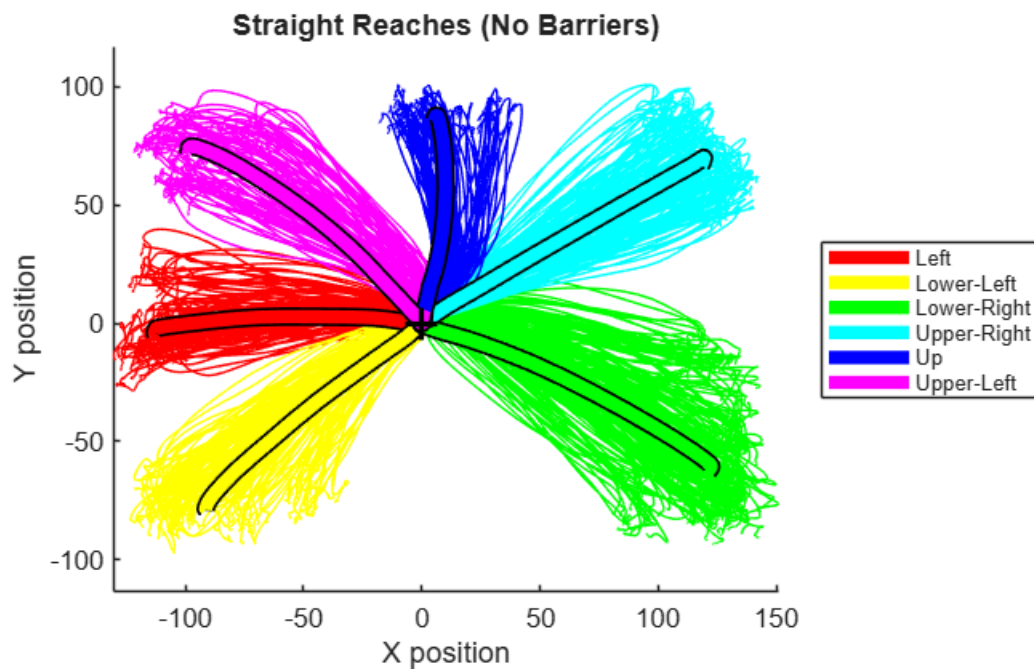


Final Project

Hand Trajectories

Behavioral: Two plots of hand position per condition over time. One of these plots should be for the curved reaches and the other should be for the straight reaches. Each direction condition will be plotted in a different color

```
plotHandTrajectories(R);
```

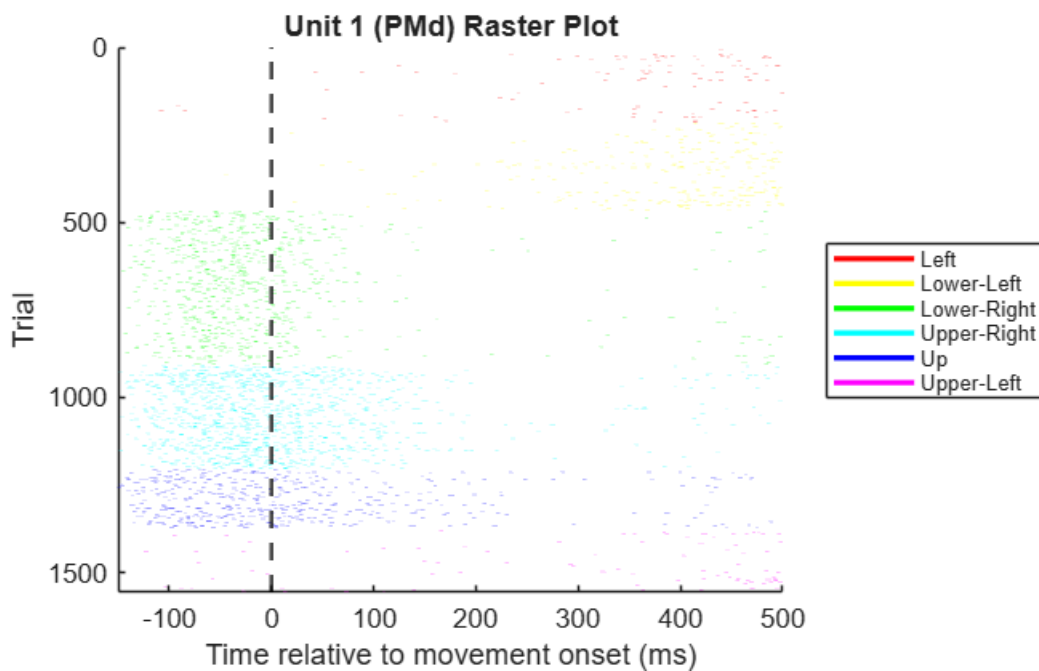


This visualization plots the hand position of the monkey throughout the trial, divided by condition (curved vs. straight) and colored by direction. The thin lines represent individual trial trajectories, whereas the bold thick lines are the trial averaged means. There are several features of the data that are worth noting. First, the data supports our reach direction clustering as a relevant task feature, as the individual trajectories naturally cluster into 6 distinct directions. Second, the two conditions produce very different kinematic trajectories. The curved reaches are noticeably more variable, reflecting the need to avoid the obstacles. On the other hand, the no barriers condition produces relatively straight and radially organized paths to their targets. Note that the mean trajectories have been time normalized. So timing differences are smoothed over for the sake of visualization.

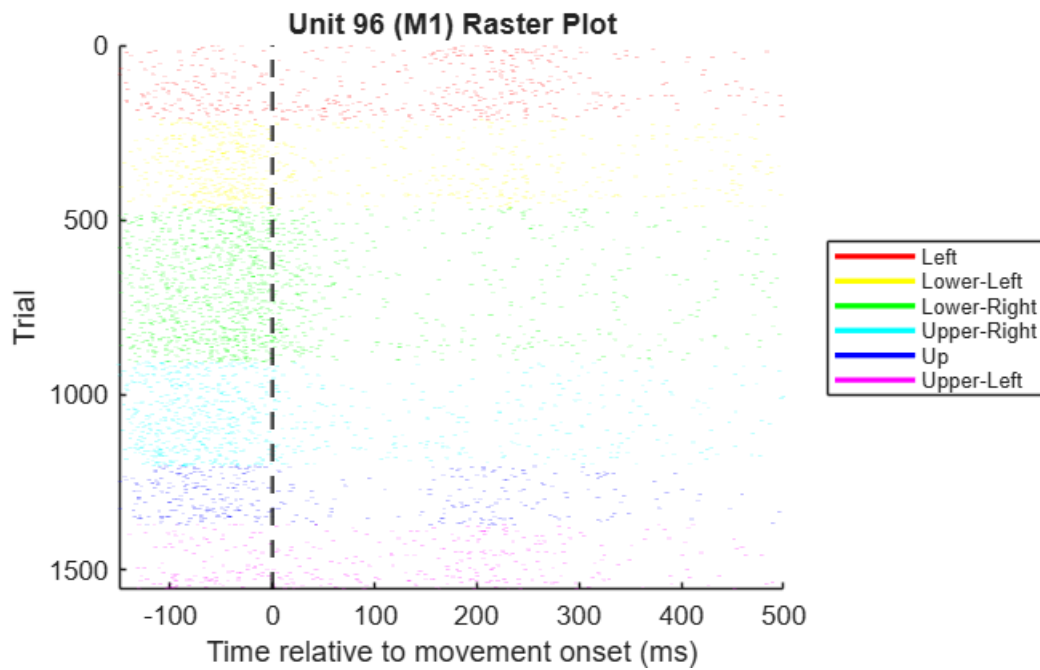
Raster Plots

Neural: Raster plot for an example M1 neuron with trials sorted by reach direction and aligned to the movement onset

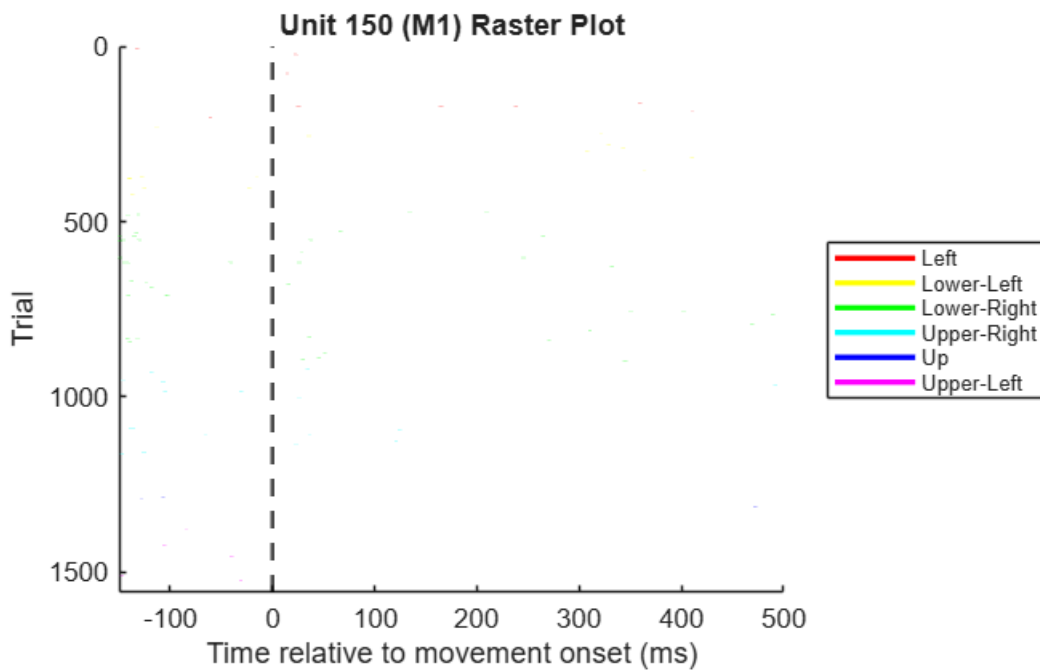
```
plotRaster(R, 1, 150, 500); % inputs in order: dataset, unit number, pre movement window(ms), post movement window(ms).
```



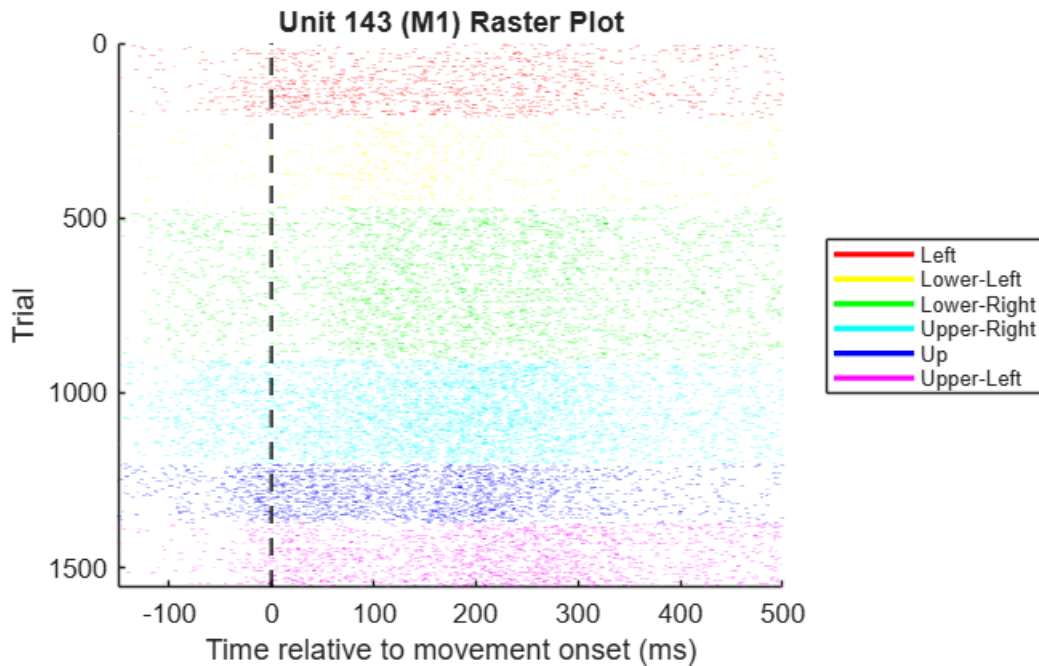
```
plotRaster(R, 96, 150, 500);
```



```
plotRaster(R, 150, 150, 500);
```



```
plotRaster(R, 143, 150, 500);
```

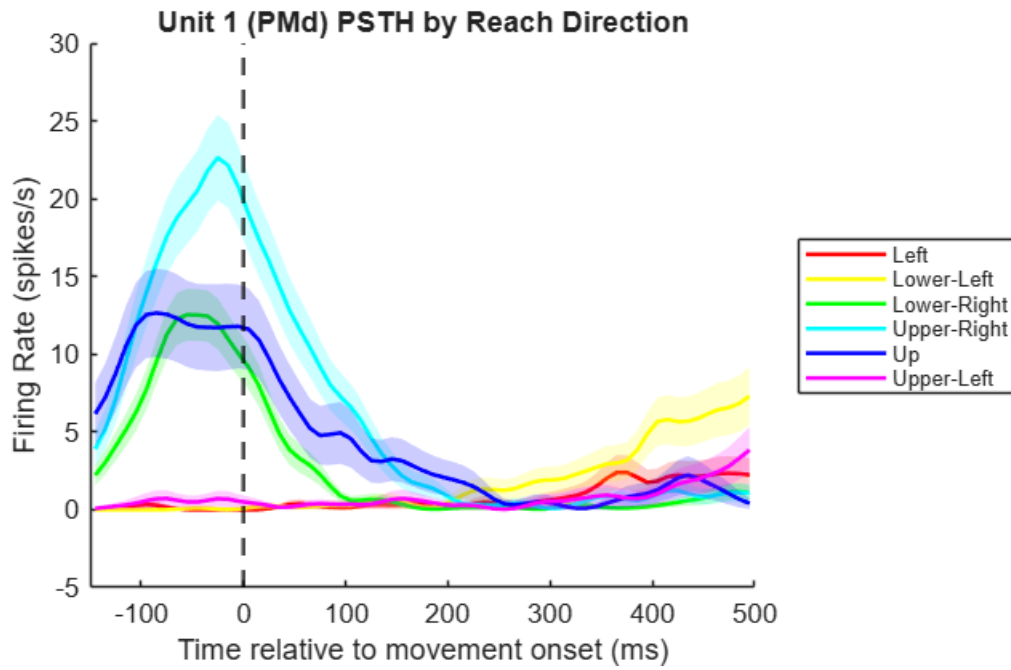


Here we've plotted several example raster plots of neural activity centered around movement onset during the task. This type of visualization allows us to characterize the neuron's directional tuning, amplitude of response, and temporal profile. We chose 4 neurons that show the heterogeneity of potential responses. Unit 1 responds predominately to reaches spanning lower right to up in the task. Notably, the peak response occurs before movement onset, which may be indicative of preparatory activity. The firing of this neuron seems to fade as the trial progresses. Unit 96 looks to have a much broader tuning profile, as it responds to nearly all reach directions, albeit a few more strongly. Similarly to neuron 1, firing seems to slow after movement onset for most of the directions, except for left/up, which has an additional response during movement. Unit 150 is not tuned to any of the directions, only sparsely firing throughout the trials with no apparent structure. Unit 143 also shows a very broad tuning profile, but it has different temporal characteristics. Activity ramps during the delay period, but peaks and maintains throughout the movement epoch. This all goes to show the variability in response in both M1 and PMd, units in both regions show different firing rates, modulation depths, and tuning.

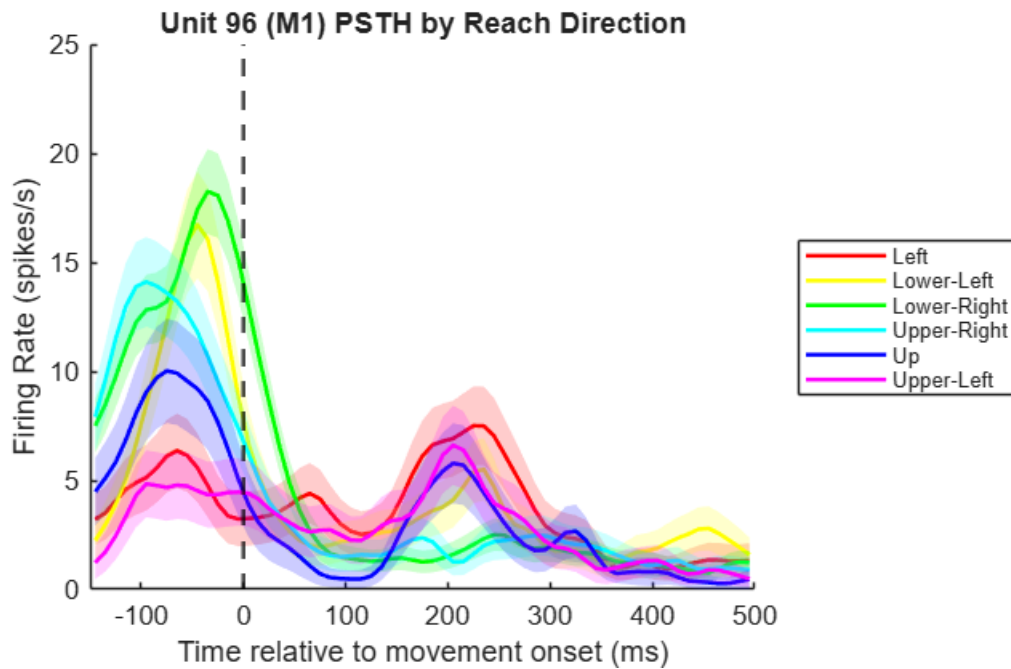
Peri-Stimulus Time Histograms

Neural: PSTH for the same example M1 neuron averaged by reach direction condition and aligned to movement onset

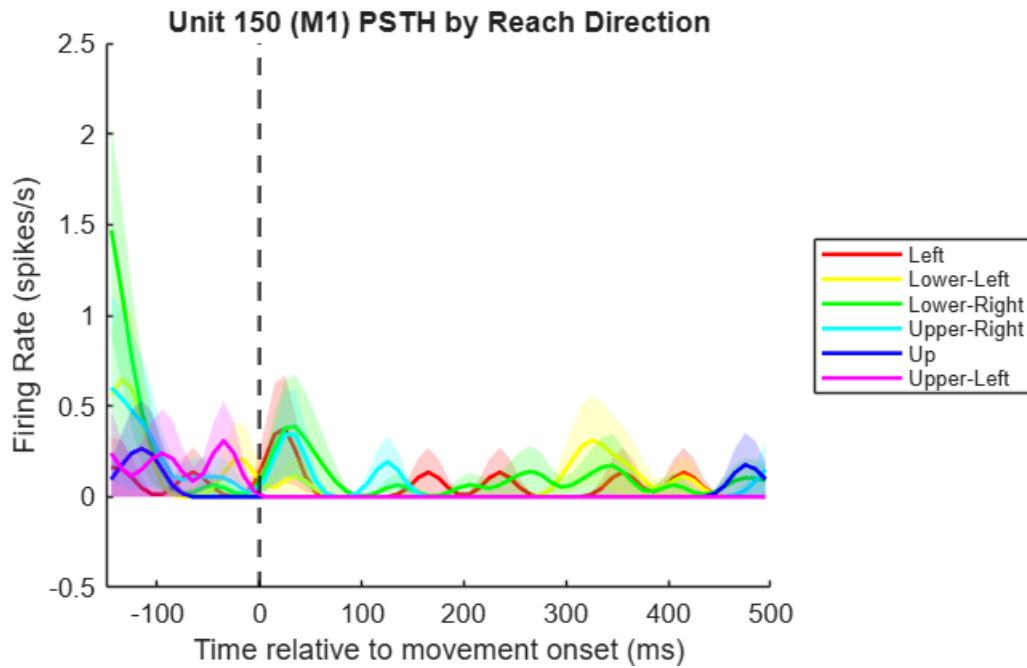
```
plotPSTH(R, 1, 150, 500, 10); % inputs in order: data set, unit number, pre
movement window(ms), post movement window(ms), bin size(ms)
```



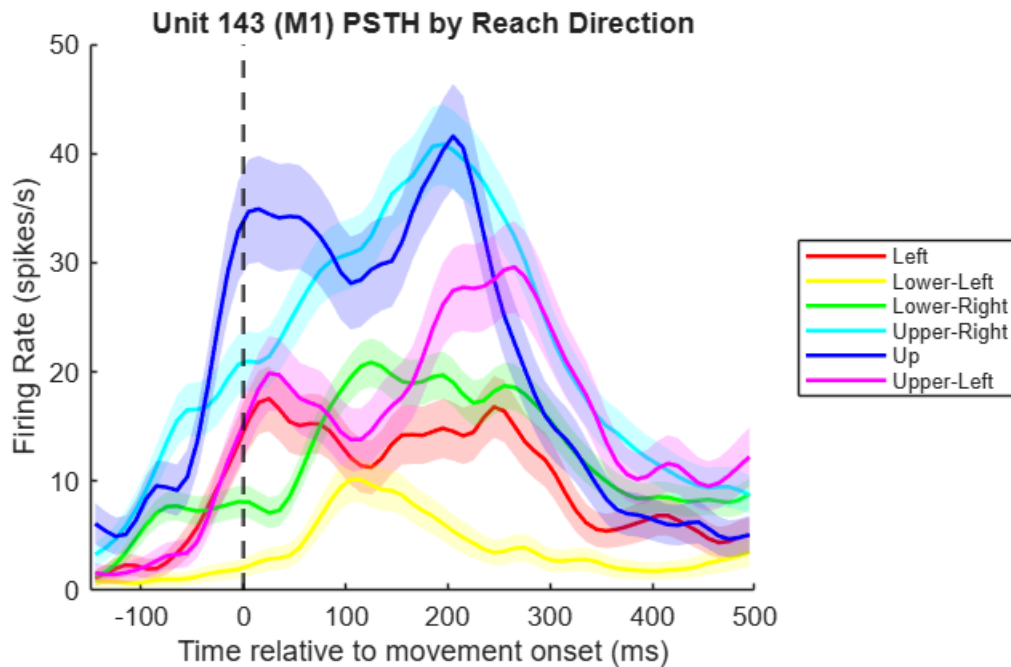
```
plotPSTH(R, 96, 150, 500, 10);
```



```
plotPSTH(R, 150, 150, 500, 10);
```



```
plotPSTH(R, 143, 150, 500, 10);
```



Here we have several Peri-Stimulus/Event Time Histograms for motor neurons aligned to movement onset. This is another type of neural data visualization that can show trends/responses across time as well as the preferred conditions where a neuron exhibits its optimal firing response.

Neuron 1: Has clear directional tuning to up through lower right, peaking before movement onset and decaying throughout the movement epoch.

Neuron 96: Has relatively strong directional tuning to most directions, with a weaker response to left and upper left. Activity peaks before movement onset and decays at a much faster rate than previously seen with neuron 1. There does seem to be an additional response at 200 ms after movement onset, which could be representative of a movement parameter or something that occurred in the trial.

Neuron 150: Shows no clear directional tuning

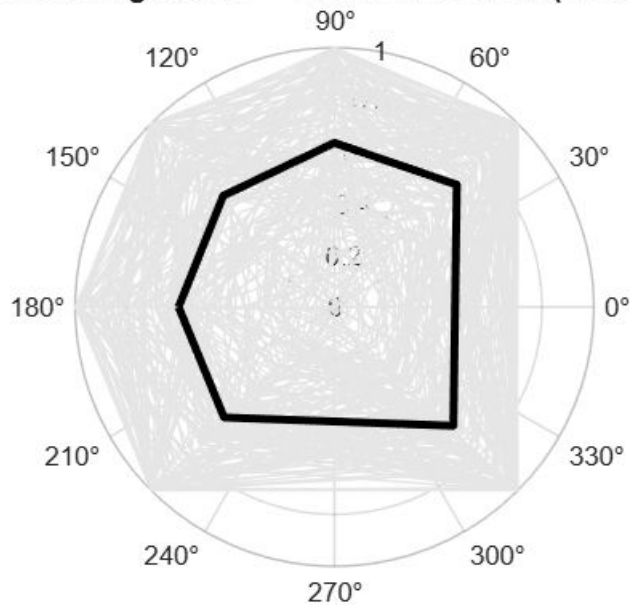
for this neuron. It's firing rate is generally flat and noisy. Neuron 143: Has broad tuning to nearly all directions. Activity ramps up during the delay period but peaks later on in the movement period around 200 ms in. Again these visualizations display the heterogeneity of response found at the level of individual units, not all M1 units are exclusively active during the movement phase and not all PMd neurons during the delay phase. Neurons in both regions can be active during both phases of the task. In comparing the PSTHs and raster plots, it often can be easier to visualize the direction that elicits the highest firing response given how the data is represented.

Tuning Curve Plot

Neural: Tuning curve plot for each neuron overlaid, with the mean in bold to show tuning across the population

```
plotTuningCurve(R);
```

Population Tuning Curves — Movement Period (n=158 neurons)



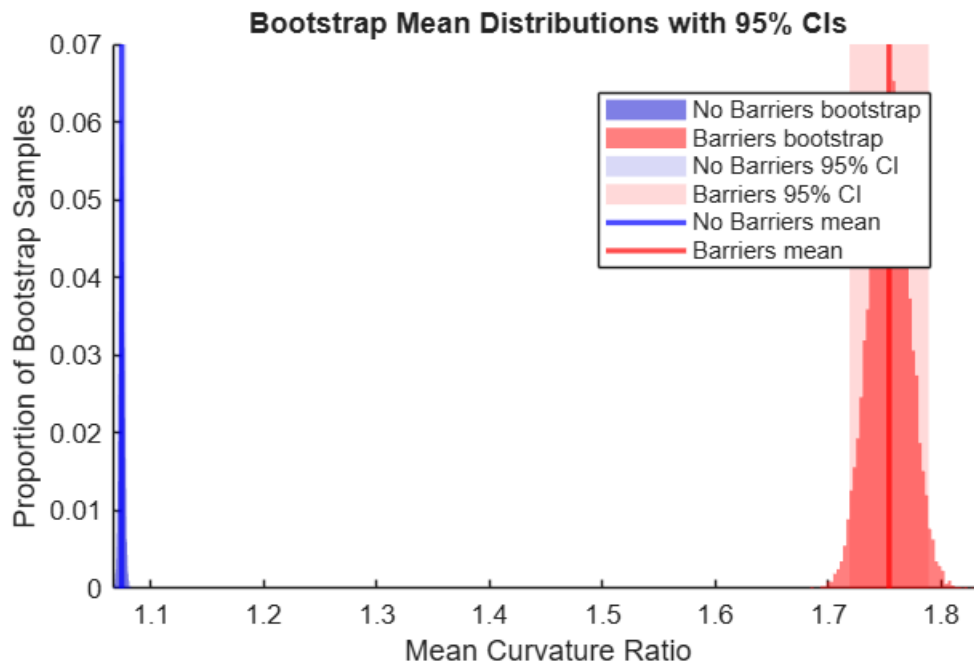
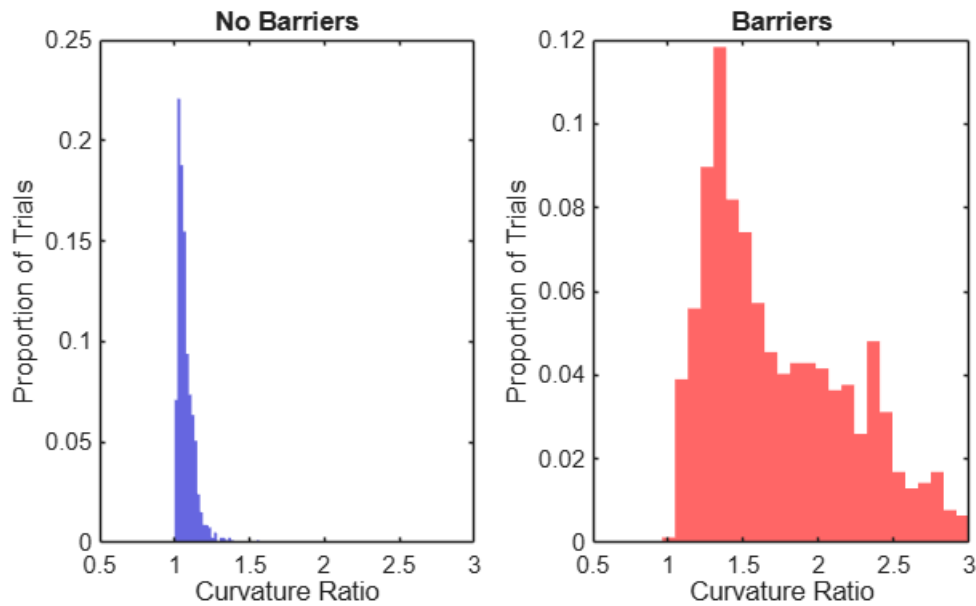
The population shows broad heterogeneity in response to all directions across both brain regions. It appears that individual neurons are tuned to all reach direction clusters, and it does not seem like one direction dominates. The population mean curve supports this as it is similarly uniform across the polar plot. This motivates population level analyses, where we can look at what the population encodes. The 6 vertices correspond to the 6 reach direction clusters we divided by. The connecting lines are thus linear interpolations between the groups.

Behavioral: Curvature Metric

Behavioral: Find a curvature metric (ratio of actual path length to straight line distance between start and end). Plot the distribution of curvature with a bootstrapped confidence interval for curved and straight path conditions. This would be a behavioral verification that the animal actually did the curved paths and not just the straight length ones.

```
curvatureMetric(R)
```

Path Curvature Distribution by Condition



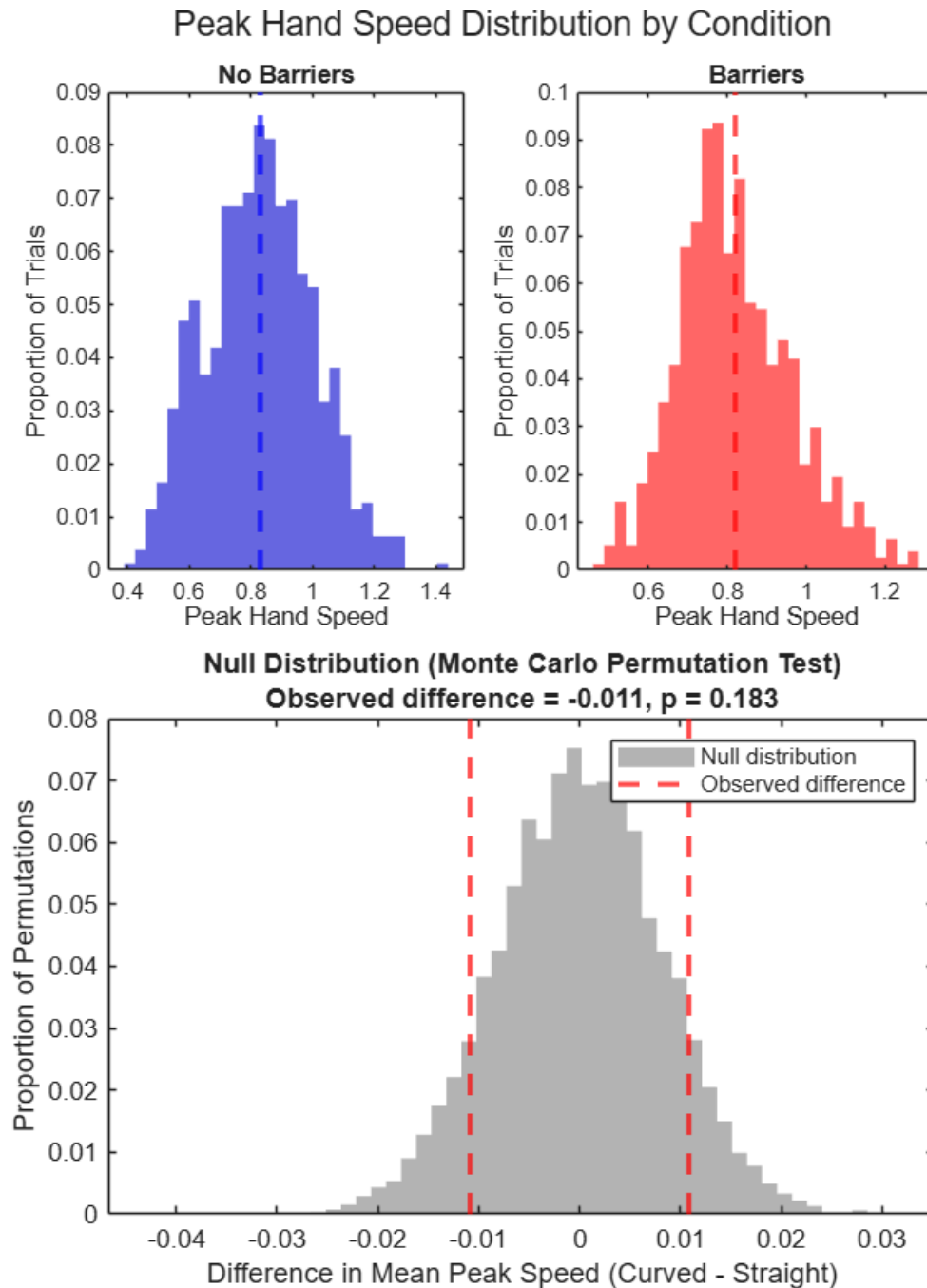
The two curvature distributions are distinguishable by the barrier and no barrier conditions. In the barriers condition, we see higher variability. This is expected as the animal needed to take an actual longer path length to complete those trials. The bootstrapped confidence interval further confirms that the curved trials were accurately performed. This non overlapping confidence intervals show us that the difference between the conditions is consistent across the population. Although we can see from the kinematic trajectory plot earlier the directions the animals took curved reaches, bootstrapping confirms that this behavior is consistent across the population of trials.

Behavioral: Peak Hand Velocity

Behavioral: Hand velocity over time split by the curved and straight reaches. Compute peak hand speed during trial, split trials into groups based on curved or straight reaches, compare the means.

Monte Carlo: shuffle the curved/straight labels, recompute the difference in the means, do this 1,000 times to create a null distribution. P-value would show us if curved and straight reaches have significantly different peak hand speeds.

```
handVelocityMonteCarlo(R)
```



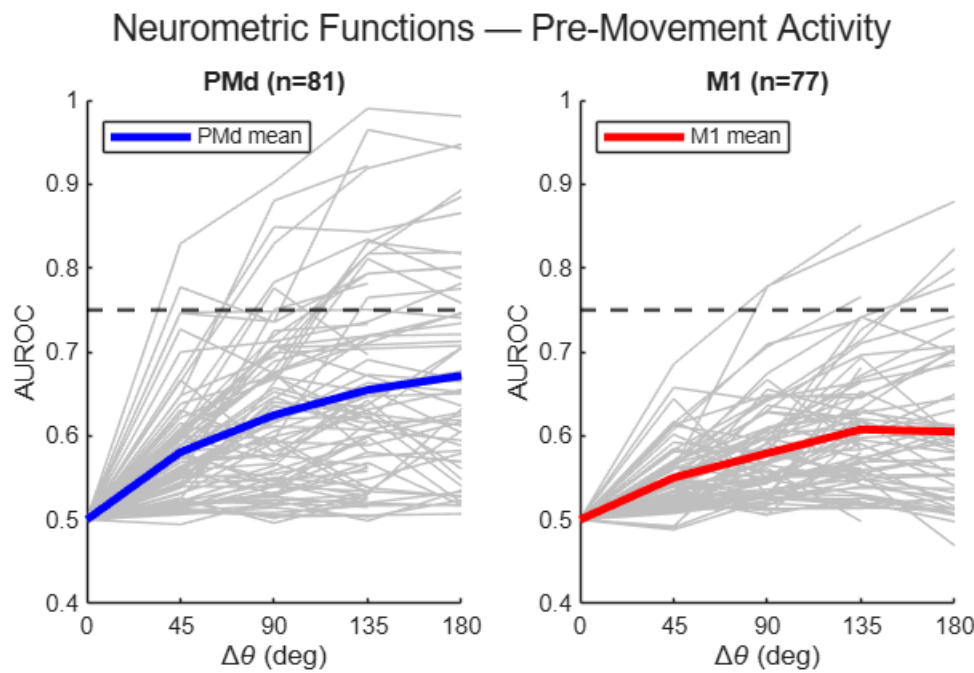
Between the barrier and no barrier conditions, the peak hand speed distributions overlap significantly. This suggests that the trajectory tasks were performed at the same speeds, reducing the confound that any variability we see in the neural data between the two conditions is due to a difference in velocity. The

differences we will see in our later analyses that separate barrier and no barrier conditions will not be due to the animal maybe moving faster in or slower in one condition type. From the Monte Carlo Permutation test, we can see that our p-value is 0.180. This means that we fail to reject the null hypothesis that the no barriers and the barriers condition differ in peak sepeed.

Traditional Single-neuron Analysis

We will use an ROC based Ideal Observer Analysis to compute each M1 neuron's discriminability between its preferred target location and each of the other targets as a function of angular distance from the preferred, producing a neurometric function per neuron. We will show the average discriminability curve across all neurons as well.

```
plotNeurometricFunctions(R, 250);
```



PMd: 18/81 neurons (22%) exceed AUROC 0.75 at any distance
M1: 6/77 neurons (8%) exceed AUROC 0.75 at any distance

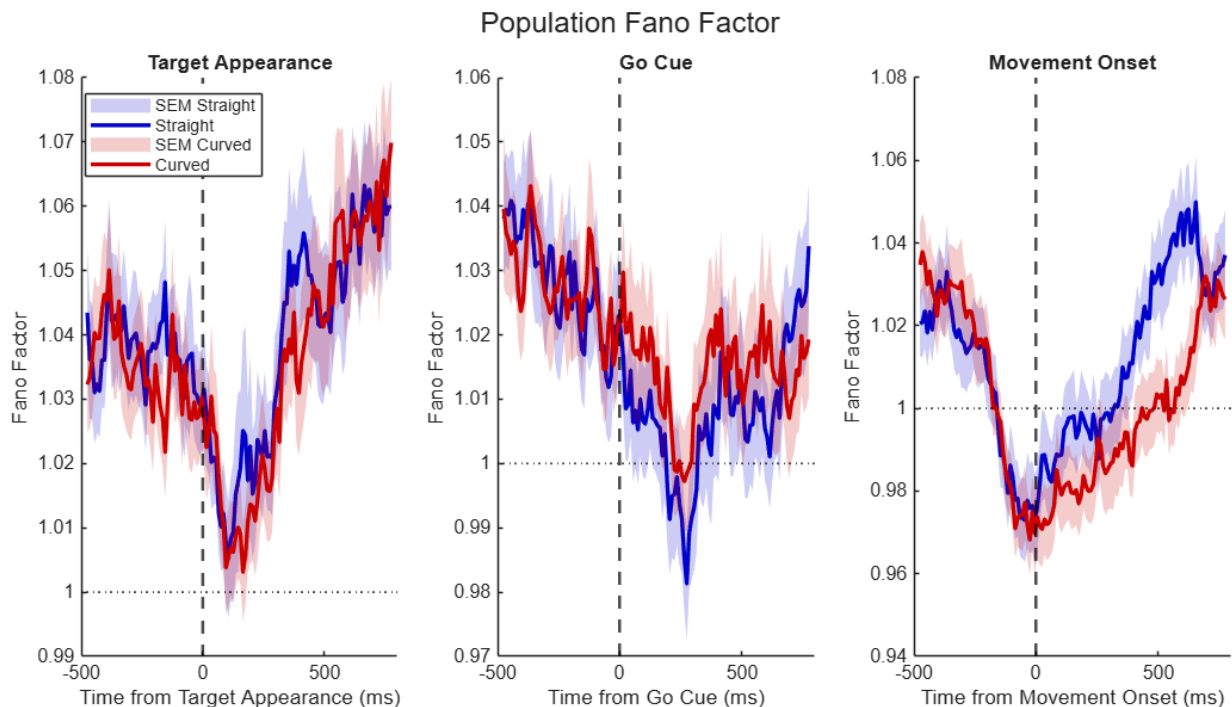
Here we have neurometric functions for all 158 motor units, split by brain region, using an ROC ideal observer analysis during the delay period of the task. We plotted each individual neuron's discriminability as a function of angular distance from its preferred reach direction as a transparent gray line, and the average curve for the population as the blue and red lines. The question addressed with this analysis is how well individual neurons can discriminate between their preferred direction and all other directions, and whether that differs across brain region. The neurometric functions reveal that PMd shows greater directional discriminability than M1 units across all angular differences, peaking at around 0.67 at opposing directions, while M1 only peaks at about 0.60. Furthermore, approximately 22% of units in PMd cross our 0.75 threshold compared to only 8% in M1. It's important to note, however, that there is a lot of heterogeneity across the population during preparatory activity. Both brain regions have neurons that are excellent discriminators, and many neurons that cluster around chance. The fact that the mean curve also never passes our threshold suggests that such task features may be better represented in population level activity. This also reveals that while PMd may have more

directional discriminability during the delay period, M1 is also able to discriminate. Thus, the functionality of these two brain regions overlap and can be thought of as a spectrum rather than a hard boundary in which PMd is exclusively motor planning and M1 is exclusively motor execution.

Trial by Trial Variability

We will compute the Fano Factor as a function of time aligned to both the target appearance, movement onset, and go-cue separately for curved and straight reaches to determine how variability evolves across delay and movement and whether this differs for the reach types, considering curved reaches are arguably more complex.

```
trialToTrialVariability(R)
```



As we learned in class, the Fano Factor (FF) is used to quantify neural variability, with $FF=1$ indicating regular firing. The plot shown was averaged At target appearance, the variability drops, and steadily rises. The same pattern is shown in the go cue alignment plot. When aligned to the movement onset, we see the straight reaches have a higher variability after movement onset compared to the curved reaches. This could mean that at movement onset, the curved trajectories execute movement in a stereotyped way. These results suggest that straight reaches are more locked to instruction, and curved reaches are more locked to execution. However, it is important to note that these differences in variability are minor. In addition, averaging over the population might minimize the differences in variability we see between the two conditions.

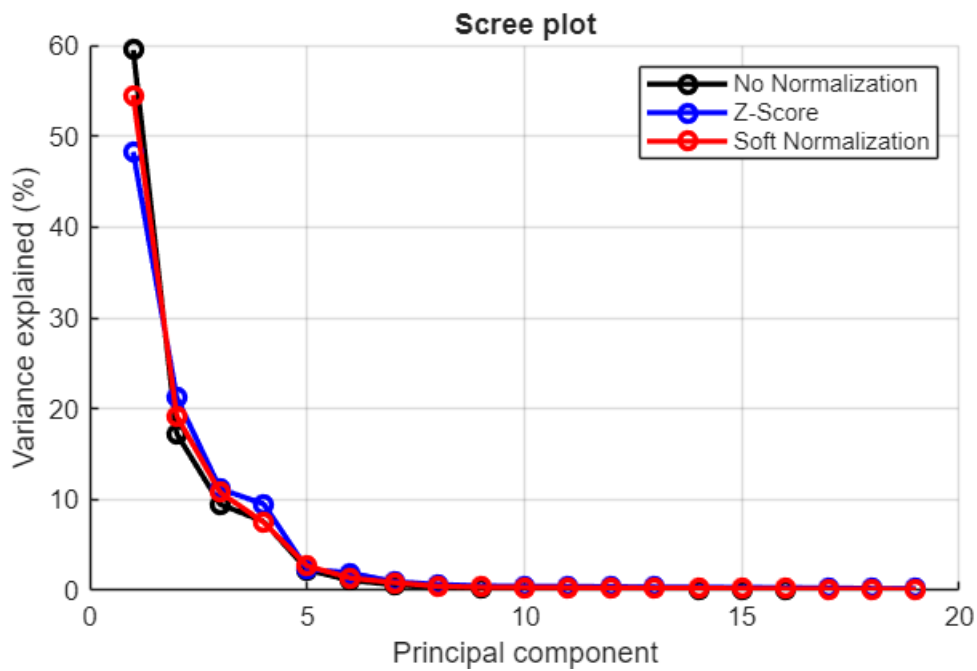
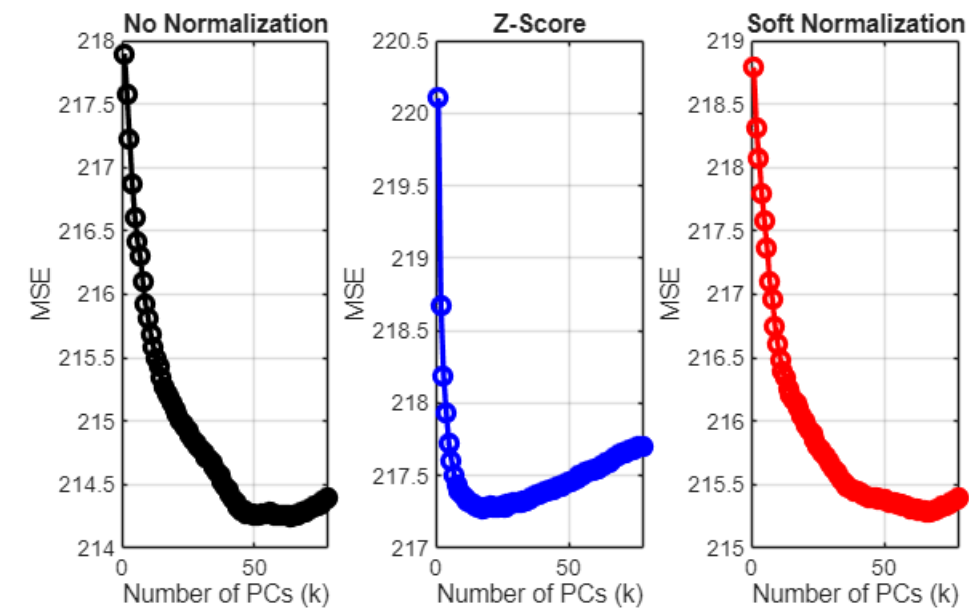
Linear Dimensionality Reduction

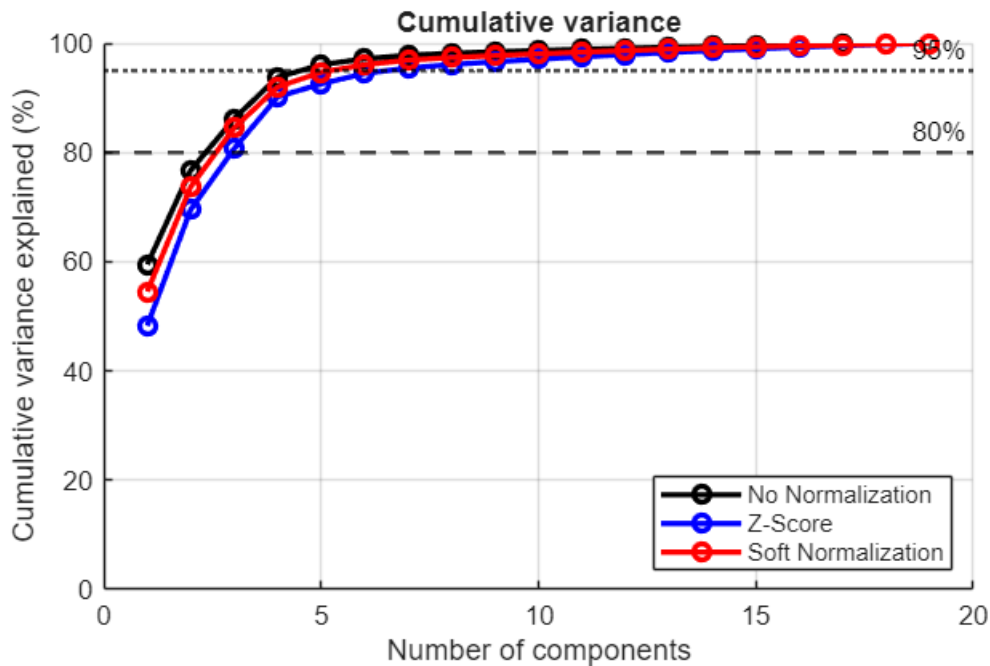
We will do cross-validated PCA on M1 activity to determine the true low dimensional structure of the neuron's responses, comparing dimensionality under no normalization, z-scoring, and soft normalization.

```
results = linearDimensionalityReduction(R);
```

Firing rates created.

Cross-validated PCA loss by dimensionality



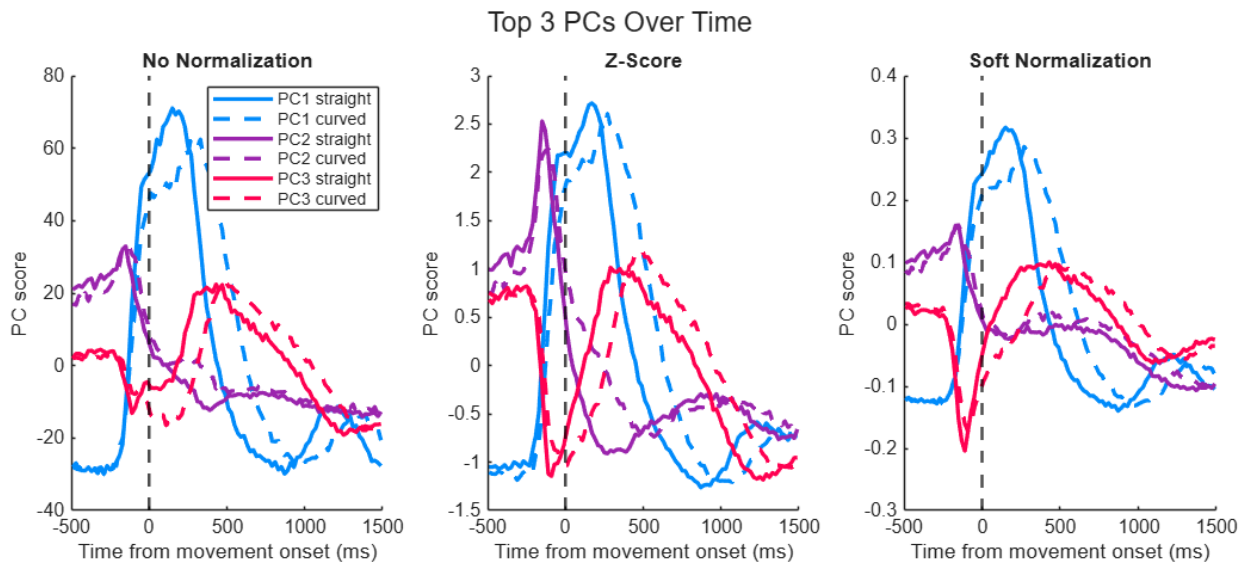


The cross-validated PCA plots comparing MSE against the number of PCs show a U-shaped curve between the different normalizations. The sharpest U-shape is in the z-scored plot. As we learned in class, this shape is expected as we increase the number of PCs and the model begins to overfit to the noise. The optimal dimensionality is therefore at the minimum of each of these plots and is around 10 components for the Z-scored plot. This optimal dimensionality varies between the different normalizations with the no normalization and soft normalization having an optimal dimensionality closer to 40 components. In our scree plot, we can see that the first few PCs account for a disproportionate amount of the variance, which is to be expected. The cumulative variance plot also confirms this as the first few components account for the majority of the variance explained.

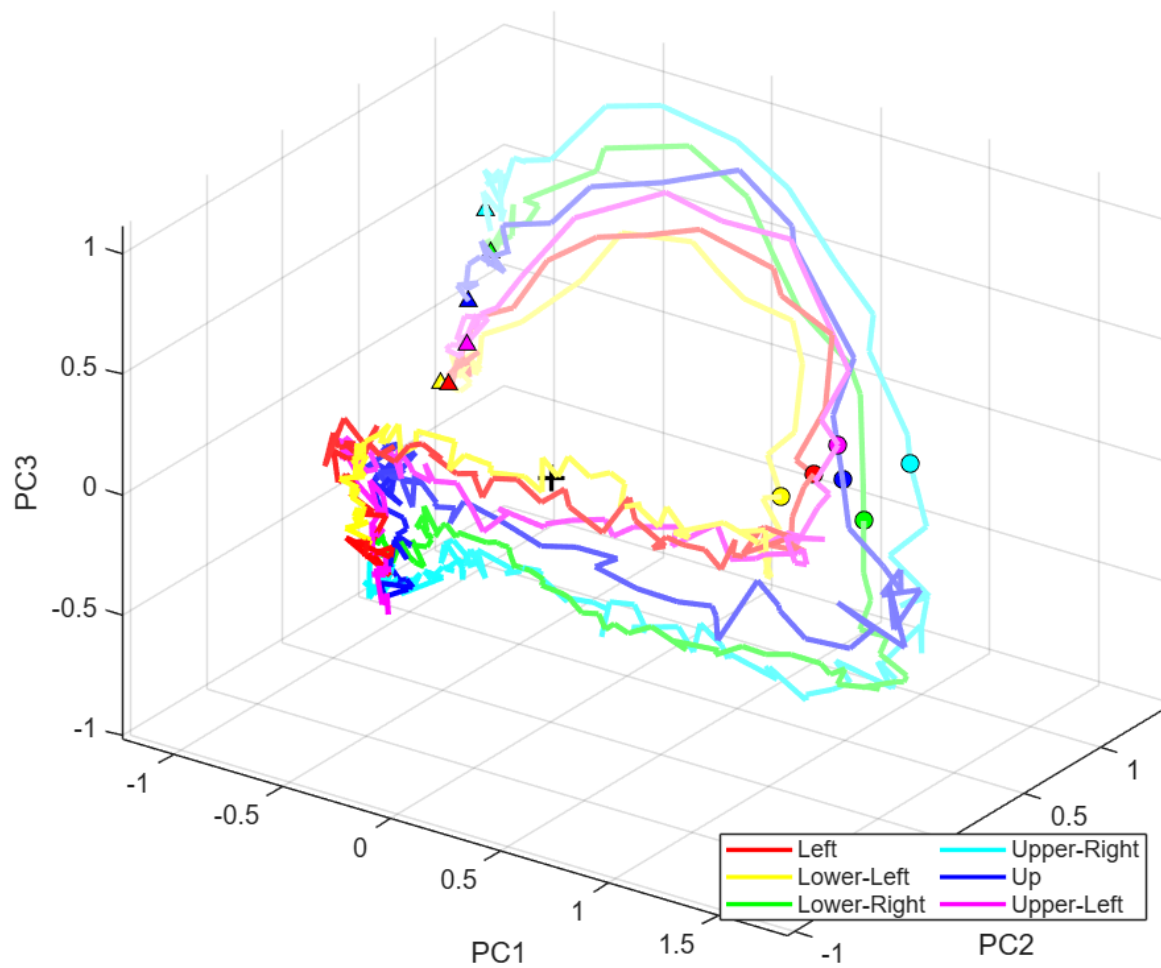
PCA Trajectory Plot

```
pcaTrajectories(R)
```

Firing rates created.



3D neural trajectories by reach direction (soft norm, all-data PCA)



▲ = start ● = movement onset

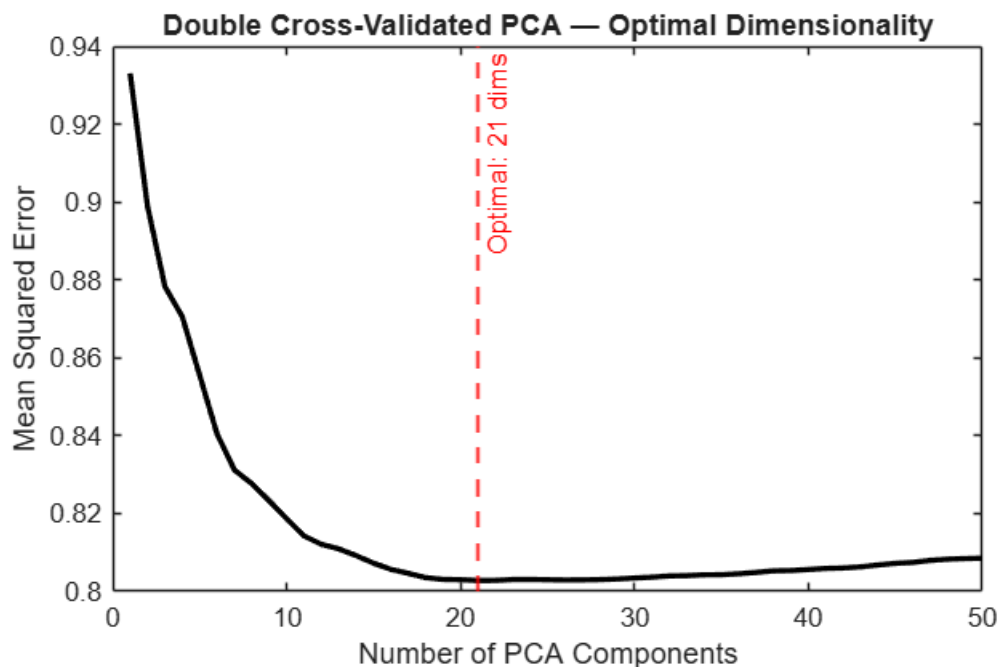
In all of these PCA trajectories, we noticed that PC1 appears to peak at movement onset. Therefore, we believe that PC1 is representative of the actual movement execution that the animal does. This peak is preserved regardless of the normalization. Based on the feedback in class, we cannot say for sure what PC2 and PC3 encode for in movement phase. We also noticed that the curved trajectories show a slight lag when compared to the straight trajectories. This might be because the curved trajectories are more complex and require more online error correction during movement execution than the straight trajectories.

When we plot the first three PCs per time in neural space, we see that for each direction the PCs follow similar trajectories. The difference seems to be in initiation point depending on what direction condition we are looking at. This suggests that there is a condition invariant signal involved in motor preparation and execution. The structure of this plot suggests that the movement direction may set an initiation point for the rest of the dynamical system to play out during the movement period.

Nonlinear Dimensionality Reduction

We will use t-SNE and UMAP on the M1 data and color their embeddings by reach conditions to determine whether both methods result in similar organization of neural states. We can follow up by correlating the pairwise distance matrices of both embeddings to compare their agreement.

```
[firingRateMatrix, conditionLabels] = buildFiringRateMatrix(R);  
[optimalDims, pcaData] = crossValidatePCA(firingRateMatrix, 50);
```



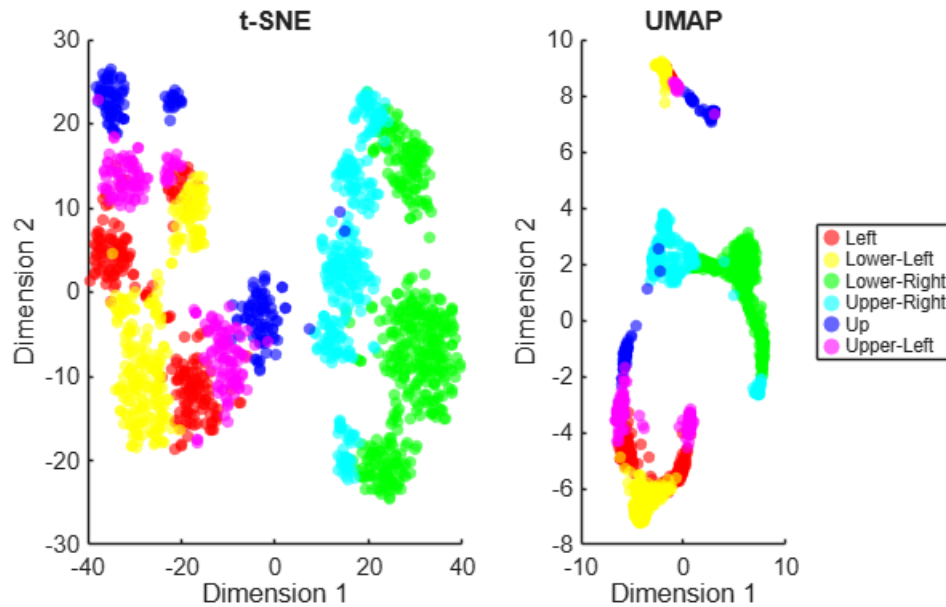
Optimal dimensionality: 21

We employed a double cross validation approach in which we split both the trials and neurons to find the optimal number of dimensions that explain the variance of our data. Upon sweeping to 50 dimensions, we found that approximately 21 dimensions minimized the mean squared error, which is the dimensionality we used in our nonlinear dimensionality reduction visualizations. This is done for the movement period of the trials. This is a necessary step to prevent tears and twists and helps deal with the "curse of dimensionality." In such

high dimension space, pairwise distances aren't reliable as nearly everything is equidistant. Running PCA on the data first, compresses it into meaningful dimensions.

```
% Plot  
plotNLDimReduction(pcaData, conditionLabels);
```

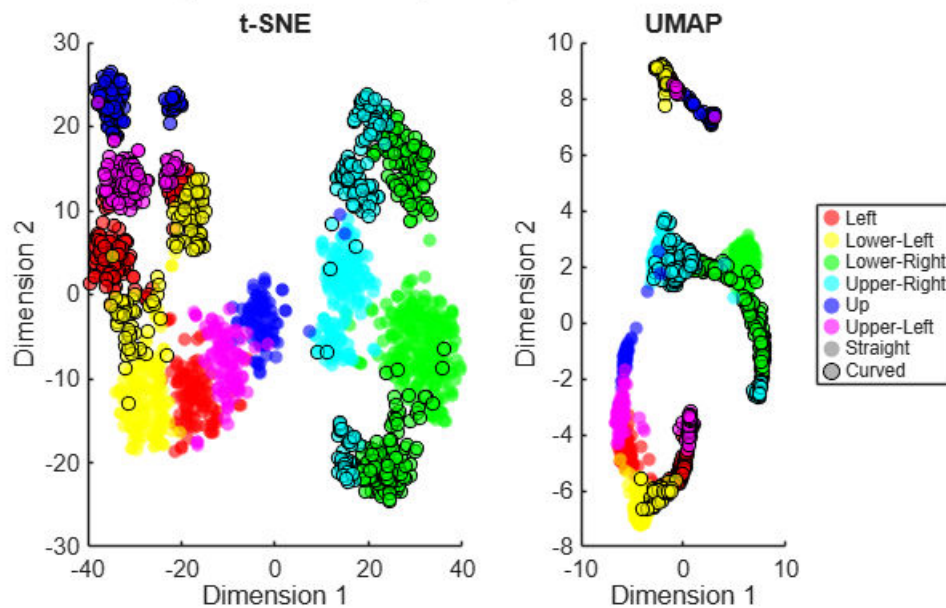
M1/PMd Population Activity — Nonlinear Dimensionality Reduction



Pairwise distance matrix correlation (t-SNE vs UMAP): $r = 0.496$

```
% Plot by Condition  
isStraight = [R.mazeCategory] == 'noBarriers';  
isStraight = isStraight(:);  
[tSNEEmbedding, umapEmbedding] = plotNLDimReductionByCondition(pcaData,  
conditionLabels, isStraight);
```

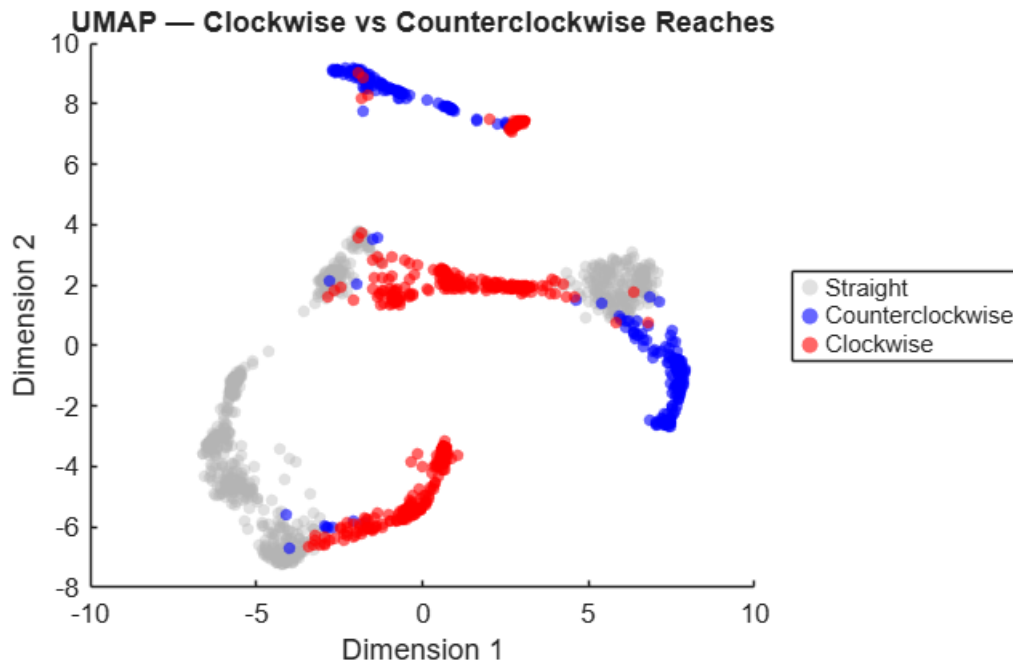

M1/PMd Population Activity — by Direction and Maze Condition



Pairwise distance matrix correlation (t-SNE vs UMAP, by-condition): $r = 0.496$
 Straight trials: 788, Curved trials: 769

Both t-SNE and UMAP separate our defined clusters, confirming that direction is a relevant axis of variance during movement. Furthermore, adjacent directions do appear to cluster in similar spaces in both methods. However, it is important to note that the geometry of each embedding should be interpreted with caution. One reason for this is that target directions are not uniformly sampled around the circle and the number of trials per direction varies a sizable amount (168 - 440). Because nonlinear dimensionality reduction can be sensitive to local neighborhood densities, the geometry of the embeddings (size and shape) may partially reflect uneven sampling rather than real neural structure. Splitting the visualization further by task condition (barrier vs no barrier) reveals that the type of reach is another feature organized in the low d manifold. This suggests that population activity also encodes reach type, something that may not be visible in individual neuron level analysis. Additional analysis: We computed the pairwise distances for every pair of trials in the embedding and then correlated the two vectors, asking the question: for each pair of trials, do t-SNE and UMAP agree on how far they are. We got $r = 0.804$, which suggests that the two methods actually agree a fair amount in their visualizations.

```
% Separate clockwise vs. counterclockwise
curveDir = getCurveDirection(R);
plotUMAPByCurve(umapEmbedding, curveDir);
```



CW: 476, CCW: 293, Straight: 788

Taking a look at whether or not the rings that emerged from the UMAP embedding were a result of counterclockwise vs clockwise reaches. We performed an additional visualization to separate the two. It appears that the rings are a product of curve direction as this feature too is well separable in the manifold. It then seems that the population activity is encoding another relevant behavioral feature in this low dimensional representation, which manifest as circular/curved patterns, similar to the movements themselves.

% Hyperparameter Examination

% UMAP

```
figure('Position', [100 100 1400 500]);

rng(1);
umap1 = umap(pcaData, 'NumNeighbors', 15, 'EmbeddingDensity', 0.1);
rng(1);
umap2 = umap(pcaData, 'NumNeighbors', 15, 'EmbeddingDensity', 10);
rng(1);
umap3 = umap(pcaData, 'NumNeighbors', 5, 'EmbeddingDensity', 0.1);

umapEmbeddings = {umap1, umap2, umap3};
umapTitles = {'UMAP (NumNeighbors=15, EmbedDensity=0.1)', ...
              'UMAP (NumNeighbors=15, EmbedDensity=10)', ...
              'UMAP (NumNeighbors=5, EmbedDensity=0.1)'};

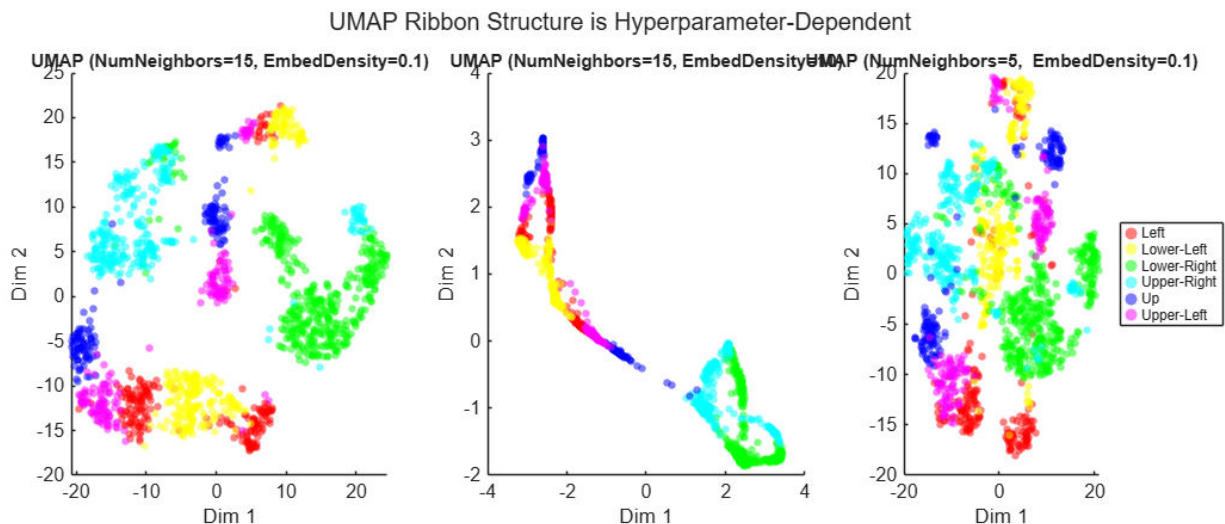
colors = hsv(6);
dirLabels = {'Left', 'Lower-Left', 'Lower-Right', 'Upper-Right', 'Up', 'Upper-Left'};
uniqueConditions = unique(conditionLabels);

for e = 1:3
    subplot(1, 3, e);
```

```

hold on;
for c = 1:6
    mask = conditionLabels == uniqueConditions(c);
    scatter(umapEmbeddings{e}(mask,1), umapEmbeddings{e}(mask,2), ...
        15, colors(c,:), 'filled', 'MarkerFaceAlpha', 0.5, ...
        'DisplayName', dirLabels{c});
end
title(umapTitles{e}, 'FontSize', 9);
xlabel('Dim 1');
ylabel('Dim 2');
if e == 3
    legend('Location', 'eastoutside', 'FontSize', 7);
end
hold off;
end
sgtitle('UMAP Ribbon Structure is Hyperparameter-Dependent', 'FontSize', 12);

```



```

% t-SNE
figure('Position', [100 100 1400 500]);

rng(1);
tsne1 = tsne(pcaData, 'NumDimensions', 2, 'Perplexity', 5, 'Standardize', true,
    'Exaggeration', 4);
rng(1);
tsne2 = tsne(pcaData, 'NumDimensions', 2, 'Perplexity', 30, 'Standardize', true,
    'Exaggeration', 4);
rng(1);
tsne3 = tsne(pcaData, 'NumDimensions', 2, 'Perplexity', 80, 'Standardize', true,
    'Exaggeration', 4);

tsneEmbeddings = {tsne1, tsne2, tsne3};
tsneTitles = {'t-SNE (Perplexity=5, Exag=4)', ...
    't-SNE (Perplexity=30, Exag=4)', ...

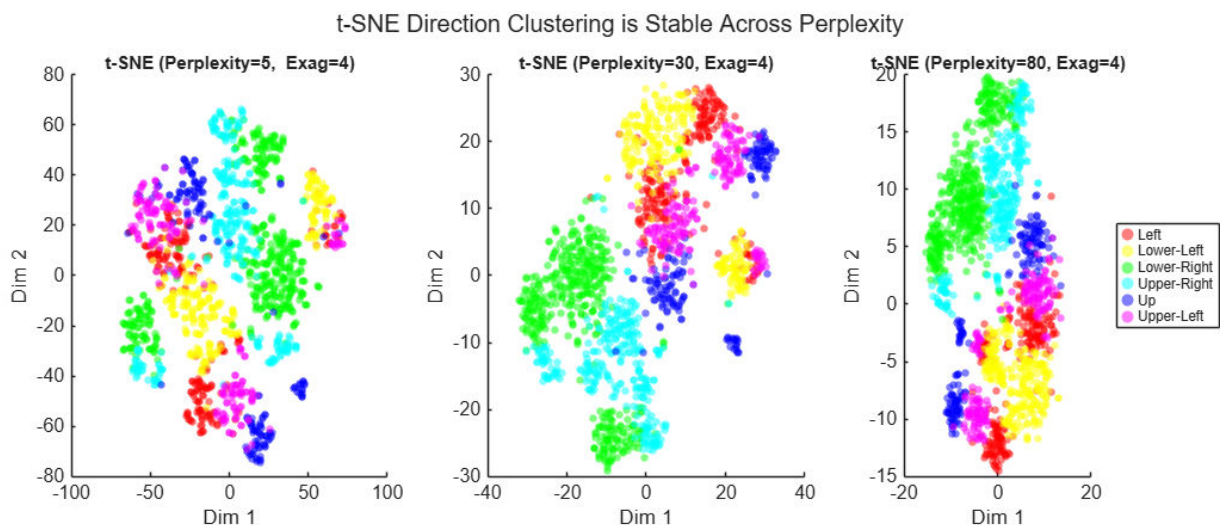
```

```

't-SNE (Perplexity=80, Exag=4)';

for e = 1:3
    subplot(1, 3, e);
    hold on;
    for c = 1:6
        mask = conditionLabels == uniqueConditions(c);
        scatter(tsneEmbeddings{e}(mask,1), tsneEmbeddings{e}(mask,2), ...
            15, colors(c,:), 'filled', 'MarkerFaceAlpha', 0.5, ...
            'DisplayName', dirLabels{c});
    end
    title(tsneTitles{e}, 'FontSize', 9);
    xlabel('Dim 1');
    ylabel('Dim 2');
    if e == 3
        legend('Location', 'eastoutside', 'FontSize', 7);
    end
    hold off;
end
sgtitle('t-SNE Direction Clustering is Stable Across Perplexity', 'FontSize', 12);

```



To test how dependent the global structure of these embeddings are on the hyperparameters, we varied perplexity for t-SNE and NumNeighbors and embedding density for UMAP. For t-SNE, perplexity controls the number of neighbors each point considers, a lower perplexity focuses on local structure, whereas a higher perplexity incorporates a broader neighborhood. Across the 3 perplexity values we tested, t-SNE produces similar looking embeddings, defined by discrete and well separated direction clusters. For UMAP, the hyperparameters we adjusted were NumNeighbors, which is similar to perplexity and embedding density, which controls how tight points are packed in the manifold. With the default settings, we saw the embedding form continuous thin structures. Increasing embedding density further compacts the visualization further. On the other hand, decreasing the number of neighbors to 5 produces a more discrete clustering as we saw in t-SNE. This reveals that the shapes we saw earlier for UMAP were largely dependent on the hyperparameters, not necessarily features of our data. Taking this together, the directional cluster remains consistent across both methods and the various parameters we tested. Trials that were towards the same reach direction tended

to group together. However, the specific differences in geometry should be treated with caution, as you can achieve relatively similar arrangements in both methods.

Decoder: Ridge Regression vs. Random Forest

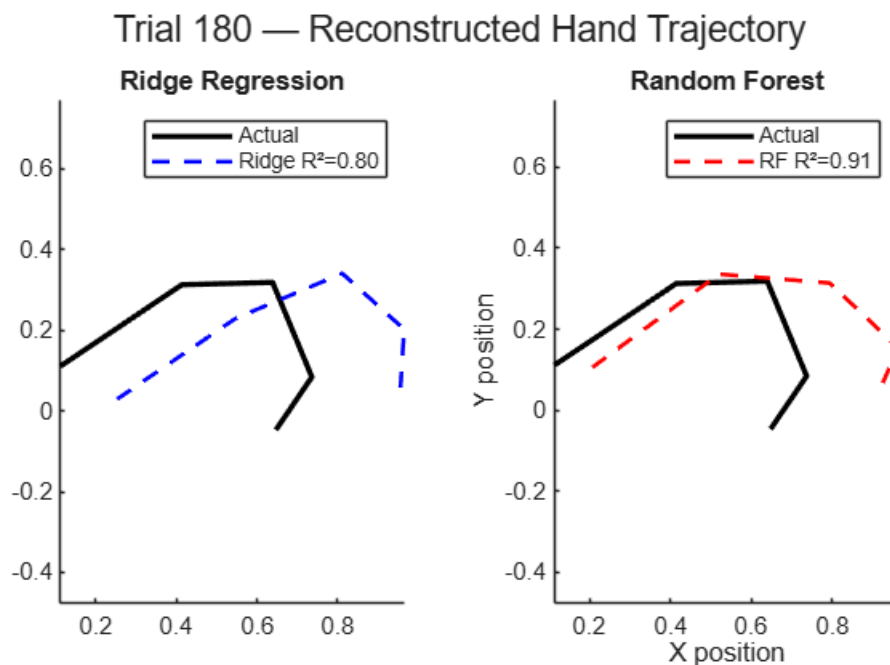
We will decode position/velocity over time from M1 neural data using 1) ridge regression and 2) random forest regression, comparing cross validated performance and their resulting reach trajectories.

```
decoderModel = trainDecoder(R, 5);
```

Ridge: best lambda: 494.1713 , CV R^2 : $V_x=0.681$, $V_y=0.470$

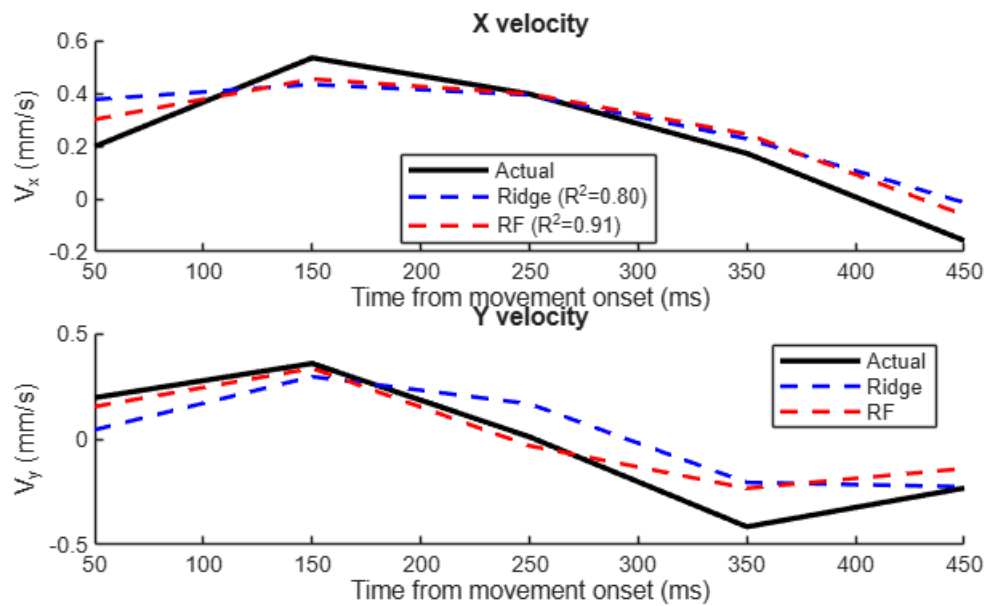
Random Forest – R^2 : $V_x=0.685$, $V_y=0.488$

```
plotDecodedTrajectory(decoderModel, R, 180);
```



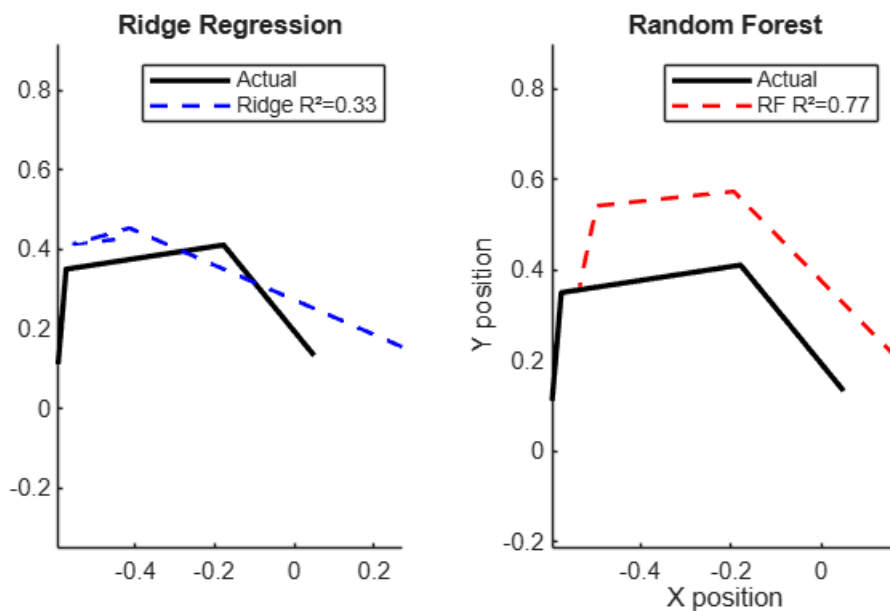
```
plotDecodedVelocity(decoderModel, R, 180);
```

Trial 180 — Actual vs Decoded Velocity



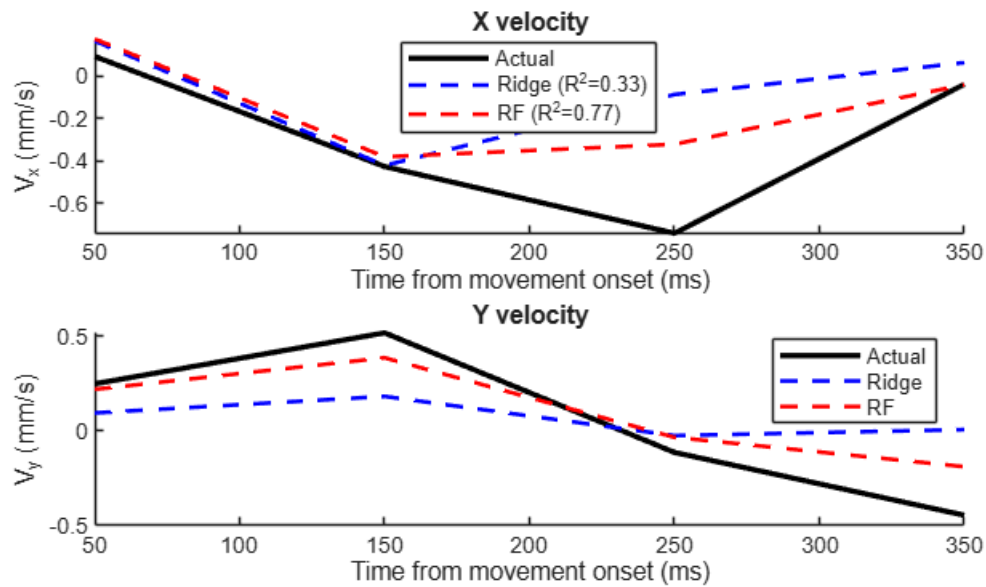
```
plotDecodedTrajectory(decoderModel, R, 300);
```

Trial 300 — Reconstructed Hand Trajectory



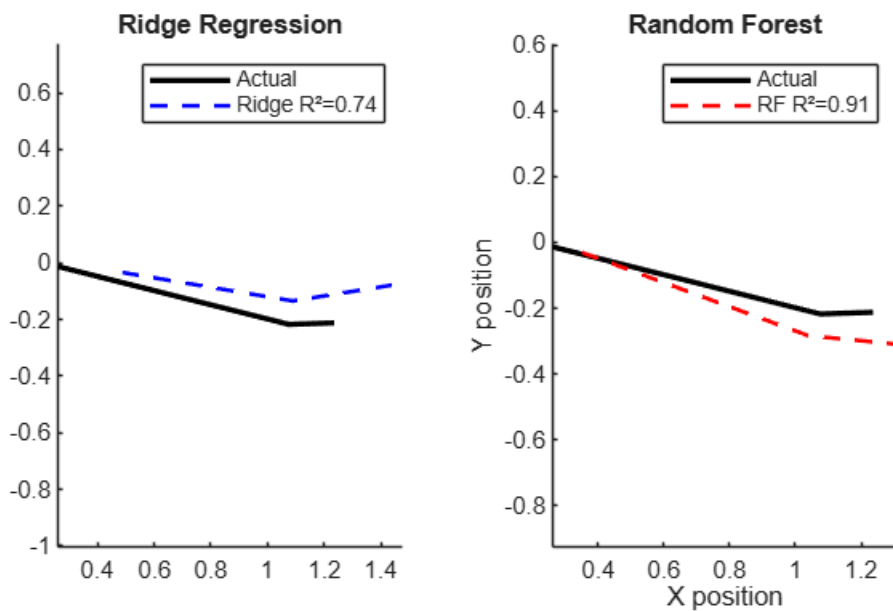
```
plotDecodedVelocity(decoderModel, R, 300)
```

Trial 300 — Actual vs Decoded Velocity



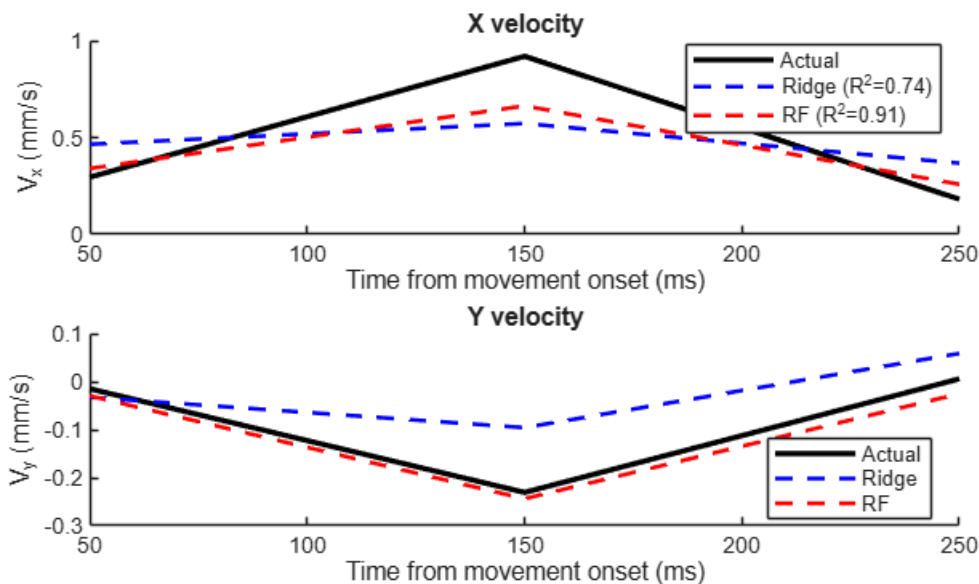
```
plotDecodedTrajectory(decoderModel, R, 131);
```

Trial 131 — Reconstructed Hand Trajectory



```
plotDecodedVelocity(decoderModel, R, 131);
```

Trial 131 — Actual vs Decoded Velocity



It appears that the nonlinear decoder(random forest) and linear decoder(ridge regression) perform similarly well, as their cross validated R^2 's are nearly identical. This lack of major difference seems to suggest that the relationship between firing rates and hand velocity is approximately linear. The difference between aggregate performance is small enough that it does not provide strong evidence of a real nonlinear relationship. On the individual trials, random forest does seem to reconstruct trajectories more faithfully, but given that we only plotted 3 trials, this is more likely just trial to trial variability. There does seem to be a tradeoff. A linear regression, like ridge regression, is more interpretable than the random forest approach, which is sort of like a black box. Random forest may perform better in some cases, but it's difficult to ask questions of. With a linear regression, we can directly ask what neurons contribute and how much, something that's important when we want to understand the motor cortex not just predict movement.

Classifier

We will classify curved vs. straight reaches based on neural data from the delay and movement period separately using SVM and logistic regression to assess when kinematic information is encoded in M1. We will assess the significance of our classifier using a Monte Carlo shuffle test, where we shuffle curved vs straight trial labels 1000 times to generate a null distribution and compute a p-value for each method/time-window combination.

```
classifier(R)
```

Firing rates computed.

Straight (No Barriers):

Value	Count	Percent
1	104	13.20%
2	133	16.88%
3	221	28.05%
4	152	19.29%
5	83	10.53%

6 95 12.06%

Curved (Barriers):

Value	Count	Percent
1	109	14.17%
2	119	15.47%
3	219	28.48%
4	146	18.99%
5	85	11.05%
6	91	11.83%

```
=====
SVM (RBF) | Delay Period | Straight (No Barriers)
=====
Observed accuracy = 0.942
p-value = 0.0010
```

```
=====
Logistic Regression | Delay Period | Straight (No Barriers)
=====
Observed accuracy = 0.939
p-value = 0.0010
```

```
=====
SVM (RBF) | Movement Period | Straight (No Barriers)
=====
Observed accuracy = 0.980
p-value = 0.0010
```

```
=====
Logistic Regression | Movement Period | Straight (No Barriers)
=====
Observed accuracy = 0.982
p-value = 0.0010
```

```
=====
SVM (RBF) | Delay Period | Curved (Barriers)
=====
Observed accuracy = 0.836
p-value = 0.0010
```

```
=====
Logistic Regression | Delay Period | Curved (Barriers)
=====
Observed accuracy = 0.840
p-value = 0.0010
```

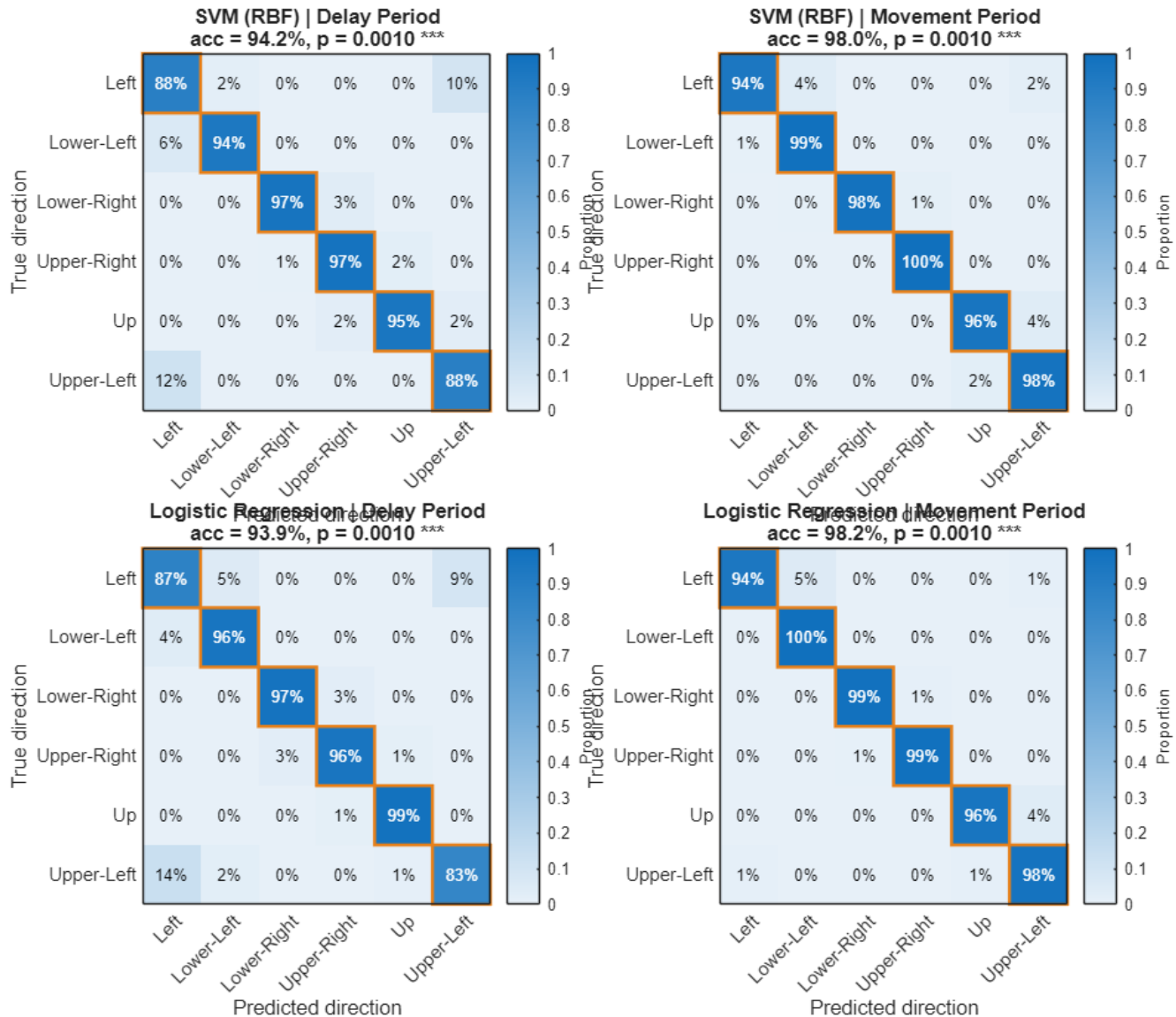
```
=====
SVM (RBF) | Movement Period | Curved (Barriers)
=====
Observed accuracy = 0.953
p-value = 0.0010
```

```
=====
Logistic Regression | Movement Period | Curved (Barriers)
=====
Observed accuracy = 0.941
p-value = 0.0010
```

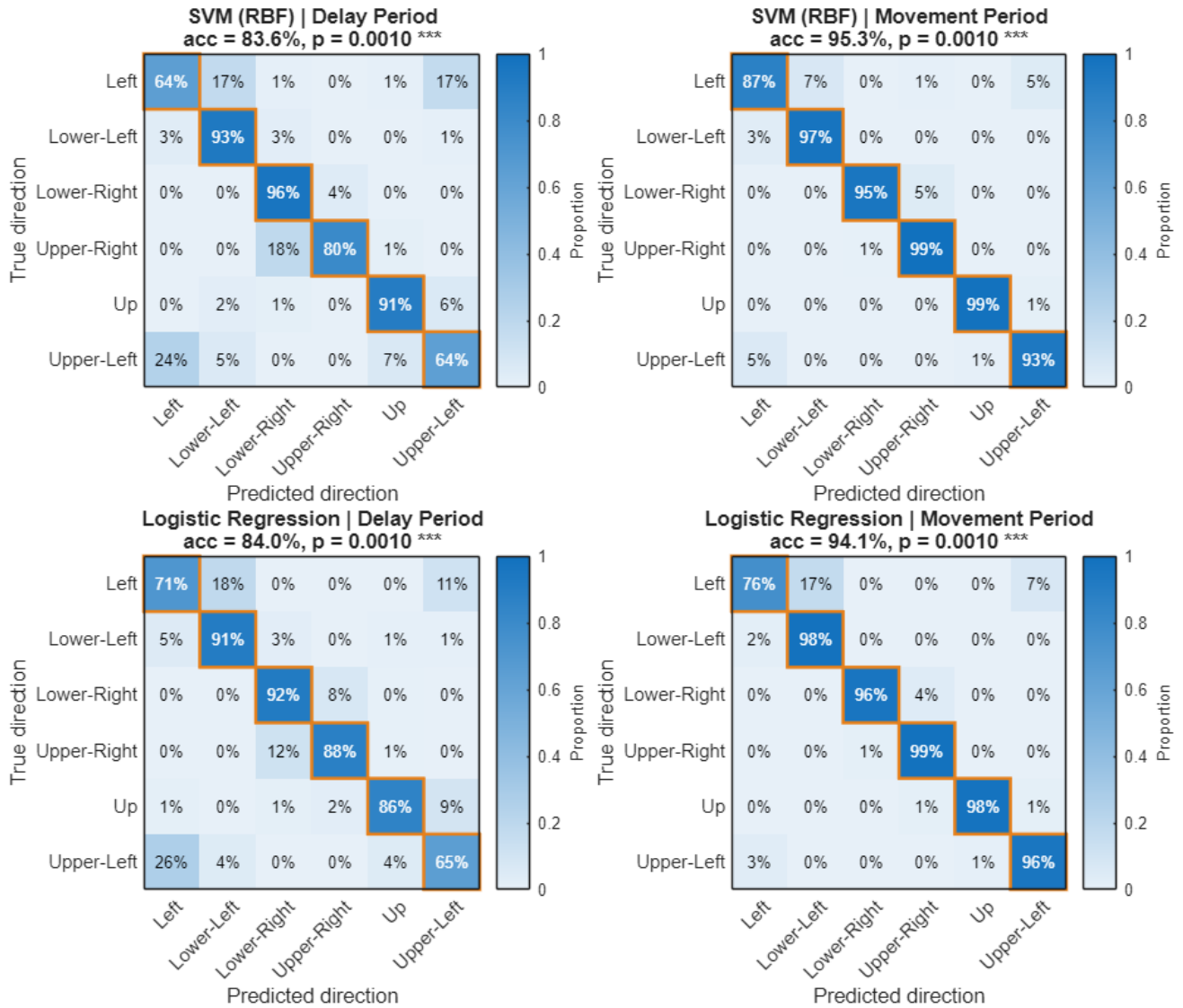
Method	Window	Condition	Accuracy	p-value	
SVM (RBF)	Delay Period	Straight (No Barriers)	0.942	0.0010	***
Logistic Regression	Delay Period	Straight (No Barriers)	0.939	0.0010	***

SVM (RBF)	Movement Period	Straight (No Barriers)	0.980	0.0010	***
Logistic Regression	Movement Period	Straight (No Barriers)	0.982	0.0010	***
SVM (RBF)	Delay Period	Curved (Barriers)	0.836	0.0010	***
Logistic Regression	Delay Period	Curved (Barriers)	0.840	0.0010	***
SVM (RBF)	Movement Period	Curved (Barriers)	0.953	0.0010	***
Logistic Regression	Movement Period	Curved (Barriers)	0.941	0.0010	***

Direction Confusion Matrix — Straight (No Barriers)



Direction Confusion Matrix — Curved (Barriers)



```
ans =
struct with fields:
  obs_acc: [2x2x2 double]
  perm_acc: [2x2x2x1000 double]
  p_values: [2x2x2 double]
  pred: {2x2x2 cell}
  true_lab: {2x2x2 cell}
  params: [1x1 struct]
```

In both the curved and straight trajectories, the classifiers performed better when trained on the movement period than on the delay period. This suggests that neural activity in the movement period more richly encodes directional information than in the delay period. In addition, both the SVM classifier with an RBF kernel and the logistic regression performed well. This might suggest that the directions are linearly separable in neural space, as both a linear and nonlinear classifier could successfully decode direction. Also, between the curved and straight trajectories, the classifier performed well. This suggests that even though the curved trajectory might have been a more complex behavior, direction is still separable.

Hand Trajectory Function

```
function plotHandTrajectories(R)
% Plot hand trajectories split by maze category
% (straight vs curved) and colored by reach direction.
% Individual trials shown as thin lines, means are the bold lines.
% All trajectories centered at movement onset position. Input is the maze
% data struc array.

% Get Direction Targets
targetXY = reshape([R.targetXY], 2, []);
angles = atan2d(targetXY(:,2), targetXY(:,1));
angleGroups = round(angles / 45) * 45;
angleGroups(angleGroups == 180) = -180;
angleGroups(angleGroups == 0 & angles < 5) = -45;
angleGroups(angleGroups == 0 & angles >= 5) = 45;
uniqueGroups = unique(angleGroups);
nGroups = length(uniqueGroups);

dirLabels = {'Left', 'Lower-Left', 'Lower-Right', 'Upper-Right', 'Up', 'Upper-
Left'};
colors = hsv(nGroups);

% Split by maze category
straightMask = [R.mazeCategory] == 'noBarriers';
curvedMask    = [R.mazeCategory] == 'barriers';
categoryMasks = {straightMask, curvedMask};
categoryNames = {'Straight Reaches (No Barriers)', 'Curved Reaches (Barriers)'};

for c = 1:2
    figure;
    hold on;

    meanXAll = cell(nGroups, 1);
    meanYAll = cell(nGroups, 1);

    for g = 1:nGroups
        groupMask = angleGroups == uniqueGroups(g);
        trialIdx  = find(groupMask & categoryMasks{c});

        if isempty(trialIdx)
            continue
        end

        allX = {};
        allY = {};

        for tr = 1:length(trialIdx)
            trIdx      = trialIdx(tr);
```

```

handX      = R(trIdx).HAND.X;
handY      = R(trIdx).HAND.Y;
handTimes  = R(trIdx).HAND.times - R(trIdx).moveOnsetTime;

% Center at movement onset position
[~, onsetIdx] = min(abs(handTimes));
handX = handX - handX(onsetIdx);
handY = handY - handY(onsetIdx);

% Plot individual trial
h = plot(handX, handY, 'Color', colors(g,:), ...
         'LineWidth', 0.5, 'HandleVisibility', 'off');
h.Color(4) = 0.05;

allX{end+1} = handX;
allY{end+1} = handY;
end

% Calculate mean
nPoints = 100;
meanX    = zeros(length(allX), nPoints);
meanY    = zeros(length(allY), nPoints);

for tr = 1:length(allX)
    tNorm      = linspace(0, 1, length(allX{tr}));
    tInterp    = linspace(0, 1, nPoints);
    meanX(tr, :) = interp1(tNorm, allX{tr}, tInterp);
    meanY(tr, :) = interp1(tNorm, allY{tr}, tInterp);
end

meanXAll{g} = mean(meanX, 1);
meanYAll{g} = mean(meanY, 1);
end

% Plot means
for g = 1:nGroups
    if ~isempty(meanXAll{g})
        plot(meanXAll{g}, meanYAll{g}, 'k-', 'LineWidth', 7, ...
             'HandleVisibility', 'off');
        plot(meanXAll{g}, meanYAll{g}, 'Color', colors(g,:), ...
             'LineWidth', 5, 'DisplayName', dirLabels{g});
    end
end

% Movement onset marker
plot(0, 0, 'k+', 'MarkerSize', 12, 'LineWidth', 2, 'HandleVisibility', 'off');

xlabel('X position');
ylabel('Y position');
title(categoryNames{c});

```

```

    legend('Location', 'eastoutside', 'FontSize', 8);
    axis equal;
    hold off;
end
end

```

Raster Plot Function

```

function plotRaster(R, unitNum, preMovMs, postMovMs)
% Plot a raster for a single motor unit aligned to movement onset,
% with trials sorted and colored by reach direction group. Inputs include:
% maze data set, unit number, pre and post movement windows.

% Get Direction Targets
targetXY = reshape([R.targetXY], 2, []);
angles = atan2d(targetXY(:,2), targetXY(:,1));
angleGroups = round(angles / 45) * 45;
angleGroups(angleGroups == 180) = -180;
angleGroups(angleGroups == 0 & angles < 5) = -45;
angleGroups(angleGroups == 0 & angles >= 5) = 45;
uniqueGroups = unique(angleGroups);
nGroups = length(uniqueGroups);

dirLabels = {'Left', 'Lower-Left', 'Lower-Right', 'Upper-Right', 'Up', 'Upper-
Left'};
colors = hsv(nGroups);

% Brain area
arrayLabel = R(1).arrays{R(1).whichArray(unitNum)};

figure;
hold on;

trialOffset = 0;

for g = 1:nGroups
    groupMask = angleGroups == uniqueGroups(g);
    groupTrials = find(groupMask);

    % Dummy line for legend
    plot(NaN, NaN, 'Color', colors(g,:), 'LineWidth', 2, ...
        'DisplayName', dirLabels{g});

    for tr = 1:length(groupTrials)
        trIdx = groupTrials(tr);

        % Align spike times to movement onset
        spks = R(trIdx).unit(unitNum).spikeTimes;
        moveOnset = R(trIdx).moveOnsetTime;
        alignedSpks = spks - moveOnset;
    end
end

```

```

% Window
windowMask = alignedSpks >= -preMovMs & alignedSpks <= postMovMs;
alignedSpks = alignedSpks(windowMask);

if isempty(alignedSpks)
    continue
end

% Plot spike ticks
y1 = trialOffset + tr;
y2 = y1 + 1;
spksRep = repmat(alignedSpks', [2 1]);
ypos = repmat([y1; y2], [1 length(alignedSpks)]);
plot(spksRep, ypos, 'Color', colors(g,:), ...
     'LineWidth', 1.5, 'HandleVisibility', 'off');
end

trialOffset = trialOffset + length(groupTrials);
end

% Movement onset line
xline(0, 'k--', 'LineWidth', 1.5, 'HandleVisibility', 'off');

set(gca, 'YDir', 'reverse', 'YLim', [0 trialOffset]);
xlabel('Time relative to movement onset (ms)');
ylabel('Trial');
xlim([-preMovMs, postMovMs]);
title(sprintf('Unit %d (%s) Raster Plot', unitNum, arrayLabel));
legend('Location', 'eastoutside', 'FontSize', 8);
hold off;
end

```

PSTH Function

```

function plotPSTH(R, unitNum, preMovMs, postMovMs, binSizeMs)
% Plot a PSTH for a motor unit aligned to movement onset,
% averaged by reach direction group with SEM error bounds. Inputs include:
% maze data set, unit to plot, ms before movement onset, ms after movement
% onset, and bin size.

% Get Direction Targets
targetXY = reshape([R.targetXY], 2, []);
angles = atan2d(targetXY(:,2), targetXY(:,1));
angleGroups = round(angles / 45) * 45;
angleGroups(angleGroups == 180) = -180;
angleGroups(angleGroups == 0 & angles < 5) = -45;
angleGroups(angleGroups == 0 & angles >= 5) = 45;
uniqueGroups = unique(angleGroups);
nGroups = length(uniqueGroups);

```

```

dirLabels = {'Left', 'Lower-Left', 'Lower-Right', 'Upper-Right', 'Up', 'Upper-Left'};
colors = hsv(nGroups);

% Time axis
binEdges = -preMovMs : binSizeMs : postMovMs;
binCenters = binEdges(1:end-1) + binSizeMs/2;
nBins = length(binCenters);

% Brain area for this unit
arrayLabel = R(1).arrays{R(1).whichArray(unitNum)};

figure;
hold on;

for g = 1:nGroups
    % Direction group trials
    groupMask = angleGroups == uniqueGroups(g);
    groupTrials = find(groupMask);
    nTrials = length(groupTrials);

    % Preallocate Matrix
    spikeCounts = zeros(nTrials, nBins);

    for tr = 1:nTrials
        trIdx = groupTrials(tr);

        % Align spike times to movement onset
        spks = R(trIdx).unit(unitNum).spikeTimes;
        moveOnset = R(trIdx).moveOnsetTime;
        alignedSpks = spks - moveOnset;

        % Bin spikes
        spikeCounts(tr, :) = histcounts(alignedSpks, binEdges);
    end

    % Convert to firing rate in spikes/s
    firingRate = spikeCounts / (binSizeMs / 1000);

    % Mean and SEM across trials
    meanFR = mean(firingRate, 1);
    semFR = std(firingRate, 0, 1) / sqrt(nTrials);
    meanFR = smoothdata(meanFR, 'gaussian', 7);
    semFR = smoothdata(semFR, 'gaussian', 7);

    % SEM error bounds
    upper = meanFR + semFR;
    lower = meanFR - semFR;
    fill([binCenters, fliplr(binCenters)], ...

```



```

        [upper, flipplr(lower)], ...
        colors(g,:), 'FaceAlpha', 0.2, 'EdgeColor', 'none', ...
        'HandleVisibility', 'off');

    % Mean line
    plot(binCenters, meanFR, 'Color', colors(g,:), 'LineWidth', 1.5, ...
        'DisplayName', dirLabels{g});
end

% Movement onset line
xline(0, 'k--', 'LineWidth', 1.5, 'HandleVisibility', 'off');

xlabel('Time relative to movement onset (ms)');
ylabel('Firing Rate (spikes/s)');
xlim([-preMovMs, postMovMs]);
title(sprintf('Unit %d (%) PSTH by Reach Direction', unitNum, arrayLabel));
legend('Location', 'eastoutside', 'FontSize', 8);
hold off;
end

```

Polar Tuning Plot Function:

```

function plotTuningCurve(R)
% Plot population tuning curves for all motor units during the movement
% period, normalized to each neuron's peak firing rate. The individual neurons are
% plotted as transparent
% lines, and the population as the bold line. Input: maze data set.

nUnits = length(R(1).unit);

% Get Direction Target
targetXY = reshape([R.targetXY], 2, []);
angles = atan2d(targetXY(:,2), targetXY(:,1));
angleGroups = round(angles / 45) * 45;
angleGroups(angleGroups == 180) = -180;
angleGroups(angleGroups == 0 & angles < 5) = -45;
angleGroups(angleGroups == 0 & angles >= 5) = 45;
uniqueGroups = unique(angleGroups);
nGroups = length(uniqueGroups);

% Convert to radians
dirRad = deg2rad(uniqueGroups);
dirRadClosed = [dirRad; dirRad(1)];

% Compute FR
allFR = zeros(nUnits, nGroups);

for u = 1:nUnits
    for g = 1:nGroups
        groupTrials = find(angleGroups == uniqueGroups(g));
    end
end

```

```

    frs = zeros(length(groupTrials), 1);

    for tr = 1:length(groupTrials)
        trIdx      = groupTrials(tr);
        spks       = R(trIdx).unit(u).spikeTimes;
        moveOnset  = R(trIdx).moveOnsetTime;
        moveEnds   = R(trIdx).moveEndsTime;
        movDur     = moveEnds - moveOnset;
        movSpks    = sum(spks >= moveOnset & spks <= moveEnds);
        frs(tr)    = movSpks / (movDur / 1000);
    end

    allFR(u, g) = mean(frs);
end
end

% Normalize
peakFR      = max(allFR, [], 2);
validUnits  = peakFR > 0;
allFRNorm   = allFR(validUnits, :) ./ peakFR(validUnits);
nValid      = sum(validUnits);

% Plotting
allFRNormClosed = [allFRNorm, allFRNorm(:, 1)];
popMean        = mean(allFRNorm, 1);
popMeanClosed  = [popMean, popMean(1)];

figure;
ax = polaraxes;
hold(ax, 'on');

% Individual neurons
for u = 1:nValid
    polarplot(ax, dirRadClosed, allFRNormClosed(u, :), ...
        'Color', [0.9 0.9 0.9], 'LineWidth', 0.3);
end

% Population mean
polarplot(ax, dirRadClosed, popMeanClosed, 'k-', 'LineWidth', 3);

title(ax, sprintf('Population Tuning Curves – Movement Period (n=%d neurons)',
    nValid));
hold(ax, 'off');
end

```

Neurometric Function

```

function plotNeurometricFunctions(R, threshold)
% Plot neurometric functions for all of the motor units using ROC-based Ideal
% Observer Analysis during the pre-movement period, split by brain area.

```

```
% Each neuron's discriminability (AUROC) is plotted as a function of
% angular distance from its preferred reach direction. Inputs: maze data
% and pre movement window length(ms)/threshold.
```

```
nUnits = length(R(1).unit);
whichArray = R(1).whichArray;
```

```
% Get Reach Targets
```

```
targetXY = reshape([R.targetXY], 2, [])';
angles = atan2d(targetXY(:,2), targetXY(:,1));
angleGroups = round(angles / 45) * 45;
angleGroups(angleGroups == 180) = -180;
angleGroups(angleGroups == 0 & angles < 5) = -45;
angleGroups(angleGroups == 0 & angles >= 5) = 45;
uniqueGroups = unique(angleGroups);
nGroups = length(uniqueGroups);
```

```
% Get differences
```

```
allAngularDiffs = zeros(nGroups, nGroups);
for g1 = 1:nGroups
    for g2 = 1:nGroups
        diff = abs(uniqueGroups(g1) - uniqueGroups(g2));
        if diff > 180
            diff = 360 - diff;
        end
        allAngularDiffs(g1, g2) = diff;
    end
end
uniqueDiffs = unique(allAngularDiffs(:));
nDiffs = length(uniqueDiffs);
```

```
% Pre-allocate matrix
```

```
aurocByDiff = zeros(nUnits, nDiffs);
```

```
for u = 1:nUnits
```

```
    meanDelayFR = zeros(nGroups, 1);
    spkCountPerGroup = cell(nGroups, 1);
```

```
    for g = 1:nGroups
        groupTrials = find(angleGroups == uniqueGroups(g));
        frs = [];
```

```
        for tr = 1:length(groupTrials)
            trIdx = groupTrials(tr);
            spks = R(trIdx).unit(u).spikeTimes;
            moveOnset = R(trIdx).moveOnsetTime;
```

```
            % Filtering
```

```
            trialAvailable = moveOnset - R(trIdx).targetAppearsTime;
```

```

        if trialAvailable > threshold
            alignedSpks = spks - moveOnset;
            preMovSpks = sum(alignedSpks >= -threshold & alignedSpks < 0);
            frs(end + 1) = preMovSpks / (threshold / 1000);
        end
    end

    meanDelayFR(g) = mean(frs);
    spkCountPerGroup{g} = frs;
end

% Find preferred direction
[~, prefIdx] = max(meanDelayFR);
prefSpkCount = spkCountPerGroup{prefIdx}';

% Compute AUROC for each direction vs preferred
aurocPerGroup = zeros(nGroups, 1);
angDiffPerGroup = zeros(nGroups, 1);

for g = 1:nGroups
    compSpkCount = spkCountPerGroup{g}';

    if g == prefIdx
        aurocPerGroup(g) = 0.5;
    else
        labels = [ones(length(prefSpkCount), 1); zeros(length(compSpkCount),
1)];

        scores = [prefSpkCount; compSpkCount];
        [~, ~, ~, auroc] = perfcurve(labels, scores, 1);
        aurocPerGroup(g) = auroc;
    end

    % Angular difference
    angDiff = abs(uniqueGroups(g) - uniqueGroups(prefIdx));
    if angDiff > 180
        angDiff = 360 - angDiff;
    end
    angDiffPerGroup(g) = angDiff;
end

% Average AUROC
for d = 1:nDiffs
    mask = angDiffPerGroup == uniqueDiffs(d);
    aurocByDiff(u, d) = mean(aurocPerGroup(mask));
end

end

% Split by area
pmdMask = whichArray == 1;
m1Mask = whichArray == 2;

```

```

areaMasks = {pmdMask, m1Mask};
areaNames = {'PMd', 'M1'};
meanColors = {'b', 'r'};

figure;
for a = 1:2
    subplot(1, 2, a);
    hold on;

    % Individual neurons
    areaUnits = find(areaMasks{a});
    for i = 1:length(areaUnits)
        u = areaUnits(i);
        validMask = ~isnan(aurocByDiff(u, :));
        plot(uniqueDiffs(validMask), aurocByDiff(u, validMask), ...
            'Color', [0.75 0.75 0.75 0.15], 'LineWidth', 0.5, ...
            'HandleVisibility', 'off');
    end

    % Population mean
    popMean = nanmean(aurocByDiff(areaMasks{a}, :), 1);
    plot(uniqueDiffs, popMean, meanColors{a}, 'LineWidth', 3, ...
        'DisplayName', sprintf('%s mean', areaNames{a}));

    % Threshold line
    yline(0.75, 'k--', 'LineWidth', 1.5, 'HandleVisibility', 'off');

    xlim([0 180]);
    xticks(0:45:180);
    ylim([0.4 1]);
    xlabel('\Delta\theta (deg)');
    ylabel('AUROC');
    title(sprintf('%s (n=%d)', areaNames{a}, sum(areaMasks{a})));
    legend('Location', 'northwest');
    hold off;
end

sgtitle('Neurometric Functions – Pre-Movement Activity');

% How many neurons crossed threshold?
threshold75 = 0.75;

pmdUnits = find(R(1).whichArray == 1);
m1Units = find(R(1).whichArray == 2);

pmdAuroc = aurocByDiff(pmdUnits, :);
m1Auroc = aurocByDiff(m1Units, :);

pmdCross = sum(any(pmdAuroc > threshold75, 2));
m1Cross = sum(any(m1Auroc > threshold75, 2));

```

```

fprintf('PMd: %d/%d neurons (%.0f%%) exceed AUROC 0.75 at any distance\n', ...
    pmdCross, length(pmdUnits), 100*pmdCross/length(pmdUnits));
fprintf('M1: %d/%d neurons (%.0f%%) exceed AUROC 0.75 at any distance\n', ...
    m1Cross, length(m1Units), 100*m1Cross/length(m1Units));
end

```

Nonlinear Dimensionality Functions:

Firing Rate Matrix

```

function [firingRateMatrix, conditionLabels] = buildFiringRateMatrix(R)
% Build a nTrials x nUnits matrix of mean firing rates during the
% full movement period for each trial, z-scored across trials. Inputs: maze
% data struc

nTrials = length(R);
nUnits = length(R(1).unit);

% Compute reach direction group for each trial
targetXY = reshape([R.targetXY], 2, []);
angles = atan2d(targetXY(:,2), targetXY(:,1));
angleGroups = round(angles / 45) * 45;
angleGroups(angleGroups == 180) = -180;
angleGroups(angleGroups == 0 & angles < 5) = -45;
angleGroups(angleGroups == 0 & angles >= 5) = 45;

% Pre-allocate
firingRateMatrix = zeros(nTrials, nUnits);

for tr = 1:nTrials
    moveOnset = R(tr).moveOnsetTime;
    moveEnds = R(tr).moveEndsTime;
    movDur = moveEnds - moveOnset;

    for u = 1:nUnits
        spks = R(tr).unit(u).spikeTimes;
        alignedSpks = spks - moveOnset;

        % Count spikes over full movement period
        movSpks = sum(alignedSpks >= 0 & alignedSpks <= movDur);
        firingRateMatrix(tr, u) = movSpks / (movDur / 1000);
    end
end

% Z-score each neuron across trials
firingRateMatrix = zscore(firingRateMatrix);

conditionLabels = angleGroups;

```

```
end
```

2x Cross Validation PCA

```
function [optimalDims, pcaData] = crossValidatePCA(firingRateMatrix, maxDims)
% Double Cross Validation to find the optimal dimensionality.
% Both observations (trials) and variables (neurons) are split into
% k folds. For each pair, we fit PCA on the
% training quadrant and evaluate the reconstruction for the held out test quadrant.

rng(1);
nTrials = size(firingRateMatrix, 1);
nUnits  = size(firingRateMatrix, 2);
nFolds  = 10;

% Trial folds
trialCv = cvpartition(nTrials, 'KFold', nFolds);

% Neuron fold
neuronPerm = randperm(nUnits);
neuronFolds = mod(0:nUnits-1, nFolds) + 1;
neuronFoldId = zeros(1, nUnits);
neuronFoldId(neuronPerm) = neuronFolds;

% Pre-allocate error matrix
mse = zeros(nFolds, nFolds, maxDims);

for trialFold = 1:nFolds
    trainTrialMask = training(trialCv, trialFold);
    testTrialMask  = test(trialCv, trialFold);

    for neuronFold = 1:nFolds
        yNeurons = find(neuronFoldId == neuronFold);
        xNeurons = find(neuronFoldId ~= neuronFold);

        % Four quadrants
        xTrain = firingRateMatrix(trainTrialMask, xNeurons);
        yTrain = firingRateMatrix(trainTrialMask, yNeurons);
        xTest  = firingRateMatrix(testTrialMask,  xNeurons);
        yTest  = firingRateMatrix(testTrialMask,  yNeurons);

        % Subtract training means
        xTrainMean = mean(xTrain, 1);
        yTrainMean = mean(yTrain, 1);
        xTrainC = xTrain - xTrainMean;
        xTestC  = xTest  - xTrainMean;
        yTrainC = yTrain - yTrainMean;

        % PCA on xTrainC, request all maxDims components at once
        [V, ~] = pca(xTrainC, 'NumComponents', maxDims);
```

```

% Project xTrain and xTest once
lowDTrain = xTrainC * V;
lowDTest  = xTestC  * V;

for d = 1:maxDims
    % Regress yTrain (centered) on top-d PC scores of xTrain
    scoresTrain = lowDTrain(:, 1:d);
    beta = (scoresTrain' * scoresTrain) \ (scoresTrain' * yTrainC);

    % Predict yTest from top-d PC scores of xTest, add back yTrainMean
    scoresTest = lowDTest(:, 1:d);
    yPred = scoresTest * beta + yTrainMean;

    mse(trialFold, neuronFold, d) = mean((yTest - yPred).^2, 'all');
end
end

% Average across both fold dimensions
meanError = squeeze(mean(mse, [1 2]));

% Plot error curve
figure;
plot(1:maxDims, meanError, 'k', 'LineWidth', 2);
xlabel('Number of PCA Components');
ylabel('Mean Squared Error');
title('Double Cross-Validated PCA – Optimal Dimensionality');

% Optimal D: minimum of the mean error curve
[~, optimalDims] = min(meanError);
xline(optimalDims, 'r--', 'LineWidth', 1.5, ...
    'Label', sprintf('Optimal: %d dims', optimalDims));

% Final PCA on all data with optimal dims
[~, pcaData] = pca(firingRateMatrix, 'NumComponents', optimalDims);

fprintf('Optimal dimensionality: %d\n', optimalDims);
end

```

NL Dimensionality Reduction - tsne and umap

```

function plotNLDimReduction(pcaData, conditionLabels)
% Apply t-SNE and UMAP to PCA-reduced neural data and plot embeddings
% colored by reach condition. Compute pairwise distance matrix
% correlation to compare the two methods.
%
% Inputs:
%   pcaData          - nTrials x nDims PCA scores
%   conditionLabels  - nTrials x 1 vector of direction group labels

```



```

uniqueConditions = unique(conditionLabels);
nConditions = length(uniqueConditions);
colors = hsv(nConditions);

dirLabels = {'Left', 'Lower-Left', 'Lower-Right', 'Upper-Right', 'Up', 'Upper-
Left'};

% Run t-SNE with PCA initialization
rng(1);
tSNEEmbedding = tsne(pcaData, 'InitialY', pcaData(:,1:2), ...
    'Perplexity', 30);

% Run UMAP
rng(1);
umapEmbedding = umap(pcaData, Reproducible=true);

% Plot side by side
figure;
embeddings = {tSNEEmbedding, umapEmbedding};
embeddingNames = {'t-SNE', 'UMAP'};

for e = 1:2
    subplot(1, 2, e);
    hold on;

    for c = 1:nConditions
        condMask = conditionLabels == uniqueConditions(c);
        scatter(embeddings{e}(condMask, 1), embeddings{e}(condMask, 2), ...
            20, colors(c,:), 'filled', 'MarkerFaceAlpha', 0.6, ...
            'DisplayName', dirLabels{c});
    end

    xlabel('Dimension 1');
    ylabel('Dimension 2');
    title(embeddingNames{e});
    hold off;
end

legend('Location', 'eastoutside', 'FontSize', 7);
sgtitle('M1/PMd Population Activity – Nonlinear Dimensionality Reduction');

% Pairwise distance matrices:
tSNEDists = pdist(tSNEEmbedding);
umapDists = pdist(umapEmbedding);
r = corr(tSNEDists', umapDists');
fprintf('Pairwise distance matrix correlation (t-SNE vs UMAP): r = %.3f\n', r);
end

```

t-SNE and UMAP - split by curved/straight

```
function [tSNEEmbedding, umapEmbedding] = plotNLDimReductionByCondition(pcaData,
conditionLabels, isStraight)
% Apply t-SNE and UMAP to PCA-reduced neural data and plot embeddings
% colored by reach direction, with maze condition encoded by marker outline
%
% Inputs:
% pcaData          - nTrials x nDims PCA scores
% conditionLabels  - nTrials x 1 vector of direction group labels
% isStraight       - nTrials x 1 vector logical of straight vs. not

uniqueConditions = unique(conditionLabels);
nConditions = length(uniqueConditions);
colors = hsv(nConditions);

dirLabels = {'Left', 'Lower-Left', 'Lower-Right', 'Upper-Right', 'Up', 'Upper-
Left'};

% t-SNE and UMAP
rng(1);
tSNEEmbedding = tsne(pcaData, 'InitialY', pcaData(:,1:2), ...
    'Perplexity', 30);
rng(1);
umapEmbedding = umap(pcaData, Reproducible=true);

figure;
embeddings = {tSNEEmbedding, umapEmbedding};
embeddingNames = {'t-SNE', 'UMAP'};

for e = 1:2
    subplot(1, 2, e);
    hold on;

    % Direction
    for c = 1:nConditions
        dirMask = conditionLabels == uniqueConditions(c);

        % Straight
        straightMask = dirMask & isStraight;
        scatter(embeddings{e}(straightMask, 1), embeddings{e}(straightMask, 2), ...
            25, colors(c,:), 'filled', 'MarkerFaceAlpha', 0.6, ...
            'DisplayName', dirLabels{c});

        % Curved
        curvedMask = dirMask & ~isStraight;
        scatter(embeddings{e}(curvedMask, 1), embeddings{e}(curvedMask, 2), ...
            25, colors(c,:), 'filled', 'MarkerEdgeColor', 'k', ...
            'LineWidth', 0.6, 'MarkerFaceAlpha', 0.6, ...
            'HandleVisibility', 'off');
```

```

end

% Dummy entries for legend
scatter(NaN, NaN, 40, [0.5 0.5 0.5], 'filled', 'MarkerFaceAlpha', 0.6, ...
    'DisplayName', 'Straight');
scatter(NaN, NaN, 40, [0.5 0.5 0.5], 'filled', 'MarkerEdgeColor', 'k', ...
    'LineWidth', 0.6, 'MarkerFaceAlpha', 0.6, 'DisplayName', 'Curved');

xlabel('Dimension 1');
ylabel('Dimension 2');
title(embeddingNames{e});
if e == 2
    legend('Location', 'eastoutside', 'FontSize', 7);
end
hold off;
end

sgtitle('M1/PMd Population Activity – by Direction and Maze Condition');

% Pairwise distance correlation
tSNEDists = pdist(tSNEEmbedding);
umapDists = pdist(umapEmbedding);
r = corr(tSNEDists', umapDists');
fprintf('Pairwise distance matrix correlation (t-SNE vs UMAP, by-condition): r = %.3f\n', r);
fprintf('Straight trials: %d, Curved trials: %d\n', ...
    sum(isStraight), sum(~isStraight));
end

```

UMAP: counterclockwise vs clockwise reaches

```

function curveDir = getCurveDirection(R)
% Classify each trial as clockwise, counterclockwise, or straight.
% Straight vs curved comes from the maze category. For curved trials, the
% sign of the path's deviation from the straight line determines CW vs CCW.

nTrials = length(R);
curveDir = strings(nTrials, 1);
isStraight = [R.mazeCategory] == 'noBarriers';

for tr = 1:nTrials
    if isStraight(tr)
        curveDir(tr) = "Straight";
        continue
    end

    % Curved trial
    hx = R(tr).HAND.X;
    hy = R(tr).HAND.Y;

```

```

ht = R(tr).HAND.times - R(tr).moveOnsetTime;
movDur = R(tr).moveEndTime - R(tr).moveOnsetTime;
mask = ht >= 0 & ht <= movDur;

x = hx(mask) - hx(find(mask, 1));
y = hy(mask) - hy(find(mask, 1));

% Straight line
targetDir = [x(end), y(end)] / norm([x(end), y(end)]);

% Trajectory relative to straight line
midPt = [x(round(end/2)), y(round(end/2))];
sideValue = targetDir(1) * midPt(2) - targetDir(2) * midPt(1);

if sideValue > 0
    curveDir(tr) = "CCW";
else
    curveDir(tr) = "CW";
end
end
end

```

Plot UMAP - split by curve direction

```

function plotUMAPByCurve(umapEmbedding, curveDir)
% Plot the UMAP embedding colored by clockwise vs counterclockwise reaches.

isCW      = curveDir == "CW";
isCCW     = curveDir == "CCW";
isStraight = curveDir == "Straight";

figure;
hold on;
scatter(umapEmbedding(isStraight, 1), umapEmbedding(isStraight, 2), ...
    20, [0.7 0.7 0.7], 'filled', 'MarkerFaceAlpha', 0.4, ...
    'DisplayName', 'Straight');
scatter(umapEmbedding(isCCW, 1), umapEmbedding(isCCW, 2), ...
    20, 'b', 'filled', 'MarkerFaceAlpha', 0.6, ...
    'DisplayName', 'Counterclockwise');
scatter(umapEmbedding(isCW, 1), umapEmbedding(isCW, 2), ...
    20, 'r', 'filled', 'MarkerFaceAlpha', 0.6, ...
    'DisplayName', 'Clockwise');

xlabel('Dimension 1');
ylabel('Dimension 2');
title('UMAP - Clockwise vs Counterclockwise Reaches');
legend('Location', 'eastoutside');
hold off;

fprintf('CW: %d, CCW: %d, Straight: %d\n', ...

```

```

sum(isCW), sum(isCCW), sum(isStraight));
end

```

Decoder Functions:

Train Decoders & Data Extraction

```

function decoderModel = trainDecoder(R, nFolds)
% Bin neural data and computes hand velocity
% for all trials, then trains ridge and random forest decoders.
% Returns both trained models in a single struc

nTrials    = length(R);
nUnits     = length(R(1).unit);
binSizeMs  = 100;

neuralData  = cell(nTrials, 1);
velocityData = cell(nTrials, 1);

for tr = 1:nTrials
    moveOnset = R(tr).moveOnsetTime;
    moveEnds  = R(tr).moveEndsTime;
    binEdges  = moveOnset : binSizeMs : moveEnds;
    nBins     = length(binEdges) - 1;

    if nBins < 1
        continue
    end

    % Bin spike counts
    trialNeural = zeros(nBins, nUnits);
    for u = 1:nUnits
        spks = R(tr).unit(u).spikeTimes;
        trialNeural(:, u) = histcounts(spks, binEdges) / (binSizeMs / 1000);
    end

    trialNeural = sqrt(trialNeural);

    % Compute hand velocity
    handTimes  = R(tr).HAND.times;
    dt         = mean(diff(handTimes));
    velX       = gradient(R(tr).HAND.X, dt);
    velY       = gradient(R(tr).HAND.Y, dt);
    binCenters = binEdges(1:end-1) + binSizeMs / 2;

    trialVel    = zeros(nBins, 2);
    trialVel(:,1) = interp1(handTimes, velX, binCenters);
    trialVel(:,2) = interp1(handTimes, velY, binCenters);
end

```

```

    neuralData{tr} = trialNeural;
    velocityData{tr} = trialVel;
end

% Train both decoders
validTrials = ~cellfun(@isempty, neuralData);
neuralData = neuralData(validTrials);
velocityData = velocityData(validTrials);

ridgeModel = trainRidgeDecoder(neuralData, velocityData, nFolds);
rfModel = trainRFDecoder(neuralData, velocityData, nFolds);

decoderModel.ridge = ridgeModel;
decoderModel.rf = rfModel;
decoderModel.binSizeMs = binSizeMs;
end

```

Ridge Regression

```

function ridgeModel = trainRidgeDecoder(neuralData, velocityData, nFolds)
% Train a ridge regression decoder to predict hand velocity (Vx, Vy)
% from firing rates. Finds optimal lambda via k-fold CV,
% then trains final model on all data.

lambdaValues = logspace(-4, 4, 50);
nLambdas = length(lambdaValues);
nTrials = length(neuralData);

cv = cvpartition(nTrials, 'KFold', nFolds);
lambdaMSE = zeros(nLambdas, 1);

for fold = 1:nFolds
    neuralTrain = vertcat(neuralData{training(cv, fold)});
    velTrain = vertcat(velocityData{training(cv, fold)});
    neuralTest = vertcat(neuralData{test(cv, fold)});
    velTest = vertcat(velocityData{test(cv, fold)});

    lambdaWeightsVx = ridge(velTrain(:,1), neuralTrain, lambdaValues, 0);
    lambdaWeightsVy = ridge(velTrain(:,2), neuralTrain, lambdaValues, 0);

    for lIdx = 1:nLambdas
        predVx = neuralTest * lambdaWeightsVx(2:end, lIdx) + lambdaWeightsVx(1, lIdx);
        predVy = neuralTest * lambdaWeightsVy(2:end, lIdx) + lambdaWeightsVy(1, lIdx);
        lambdaMSE(lIdx) = lambdaMSE(lIdx) + ...
            (mean((velTest(:,1) - predVx).^2) + mean((velTest(:,2) - predVy).^2)) / 2;
    end
end

```

```

lambdaMSE    = lambdaMSE / nFolds;
[~, bestIdx] = min(lambdaMSE);
bestLambda   = lambdaValues(bestIdx);

% CV and R^2
foldR2 = zeros(nFolds, 2);
for fold = 1:nFolds
    neuralTrain = vertcat(neuralData{training(cv, fold)});
    velTrain    = vertcat(velocityData{training(cv, fold)});
    neuralTest  = vertcat(neuralData{test(cv, fold)});
    velTest     = vertcat(velocityData{test(cv, fold)});

    wVx = ridge(velTrain(:,1), neuralTrain, bestLambda, 0);
    wVy = ridge(velTrain(:,2), neuralTrain, bestLambda, 0);

    predVx = neuralTest * wVx(2:end) + wVx(1);
    predVy = neuralTest * wVy(2:end) + wVy(1);

    foldR2(fold,1) = 1 - sum((velTest(:,1) - predVx).^2) / sum((velTest(:,1) -
mean(velTest(:,1))).^2);
    foldR2(fold,2) = 1 - sum((velTest(:,2) - predVy).^2) / sum((velTest(:,2) -
mean(velTest(:,2))).^2);
end

r2 = mean(foldR2, 1);
fprintf('Ridge: best lambda: %.4f , CV R^2: Vx=%.3f, Vy=%.3f\n', bestLambda, r2(1),
r2(2));

% Train model on all data
neuralAll = vertcat(neuralData{:});
velAll    = vertcat(velocityData{:});

finalWeightsVx = ridge(velAll(:,1), neuralAll, bestLambda, 0);
finalWeightsVy = ridge(velAll(:,2), neuralAll, bestLambda, 0);

ridgeModel.beta      = [finalWeightsVx, finalWeightsVy];
ridgeModel.lambda    = bestLambda;
ridgeModel.r2        = r2;
ridgeModel.lambdaMSE  = lambdaMSE;
ridgeModel.lambdaValues = lambdaValues;
end

```

Random Forest:

```

function rfModel = trainRFDecoder(neuralData, velocityData, nFolds)
% Train a random forest decoder to predict hand velocity (Vx, Vy)
% from firing rates. Gets R^2 from k-fold CV, and
% then trains final models on all data.

```

```

nTrials = length(neuralData);
nTrees = 100;
cv = cvpartition(nTrials, 'KFold', nFolds);
foldR2 = zeros(nFolds, 2);

for fold = 1:nFolds
    neuralTrain = vertcat(neuralData{training(cv, fold)});
    velTrain = vertcat(velocityData{training(cv, fold)});
    neuralTest = vertcat(neuralData{test(cv, fold)});
    velTest = vertcat(velocityData{test(cv, fold)});

    rfVx = TreeBagger(nTrees, neuralTrain, velTrain(:,1), 'Method', 'regression',
'OOBPrediction', 'off');
    rfVy = TreeBagger(nTrees, neuralTrain, velTrain(:,2), 'Method', 'regression',
'OOBPrediction', 'off');
    yPred = [predict(rfVx, neuralTest), predict(rfVy, neuralTest)];

    foldR2(fold,1) = 1 - sum((velTest(:,1) - yPred(:,1)).^2) / sum((velTest(:,1) -
mean(velTest(:,1))).^2);
    foldR2(fold,2) = 1 - sum((velTest(:,2) - yPred(:,2)).^2) / sum((velTest(:,2) -
mean(velTest(:,2))).^2);
end

meanR2 = mean(foldR2, 1);
fprintf('Random Forest - R2: Vx=%.3f, Vy=%.3f\n', meanR2(1), meanR2(2));

% Train model on all data
neuralAll = vertcat(neuralData{:});
velAll = vertcat(velocityData{:});

finalRFVx = TreeBagger(nTrees, neuralAll, velAll(:,1), 'Method', 'regression',
'OOBPrediction', 'off');
finalRFVy = TreeBagger(nTrees, neuralAll, velAll(:,2), 'Method', 'regression',
'OOBPrediction', 'off');

rfModel.rfVx = finalRFVx;
rfModel.rfVy = finalRFVy;
rfModel.nTrees = nTrees;
rfModel.r2 = meanR2;
end

```

Trajectory Plot

```

function plotDecodedTrajectory(decoderModel, R, trialNum)
% Plot actual vs decoded hand trajectory for a chosen trial.
% Uses trained ridge and random forest models from trainDecoder.
% Trajectories are normalized to unit path length.

binSizeMs = decoderModel.binSizeMs;
nUnits = length(R(1).unit);

```



```

moveOnset = R(trialNum).moveOnsetTime;
moveEnds  = R(trialNum).moveEndsTime;
binEdges  = moveOnset : binSizeMs : moveEnds;
nBins     = length(binEdges) - 1;

trialNeural = zeros(nBins, nUnits);
for u = 1:nUnits
    spks = R(trialNum).unit(u).spikeTimes;
    trialNeural(:, u) = histcounts(spks, binEdges) / (binSizeMs / 1000);
end

trialNeural = sqrt(trialNeural);

ridgeVel      = zeros(nBins, 2);
ridgeVel(:,1) = trialNeural * decoderModel.ridge.beta(2:end,1) +
decoderModel.ridge.beta(1,1);
ridgeVel(:,2) = trialNeural * decoderModel.ridge.beta(2:end,2) +
decoderModel.ridge.beta(1,2);

rfVel = [predict(decoderModel.rf.rfVx, trialNeural), ...
predict(decoderModel.rf.rfVy, trialNeural)];

handTimes = R(trialNum).HAND.times;
dt         = mean(diff(handTimes));
velX       = gradient(R(trialNum).HAND.X, dt);
velY       = gradient(R(trialNum).HAND.Y, dt);
binCenters = binEdges(1:end-1) + binSizeMs / 2;

actualVel   = zeros(nBins, 2);
actualVel(:,1) = interp1(handTimes, velX, binCenters);
actualVel(:,2) = interp1(handTimes, velY, binCenters);

% Trial R^2
r2Ridge = 1 - sum((actualVel(:) - ridgeVel(:)).^2) / sum((actualVel(:) -
mean(actualVel(:))).^2);
r2RF     = 1 - sum((actualVel(:) - rfVel(:)).^2) / sum((actualVel(:) -
mean(actualVel(:))).^2);

dtSec     = binSizeMs / 1000;
actualTraj = normalizeTrajectory(cumsum(actualVel * dtSec, 1));
ridgeTraj  = normalizeTrajectory(cumsum(ridgeVel * dtSec, 1));
rfTraj     = normalizeTrajectory(cumsum(rfVel * dtSec, 1));

figure;
subplot(1,2,1); hold on;
plot(actualTraj(:,1), actualTraj(:,2), 'k-', 'LineWidth', 2, 'DisplayName',
'Actual');
plot(ridgeTraj(:,1), ridgeTraj(:,2), 'b--', 'LineWidth', 1.5, 'DisplayName',
sprintf('Ridge R²=%.2f', r2Ridge));

```

```

title('Ridge Regression'); legend; axis equal; hold off;

subplot(1,2,2); hold on;
plot(actualTraj(:,1), actualTraj(:,2), 'k-', 'LineWidth', 2, 'DisplayName',
'Actual');
plot(rfTraj(:,1), rfTraj(:,2), 'r--', 'LineWidth', 1.5, 'DisplayName',
sprintf('RF R²=%.2f', r2RF));
xlabel('X position'); ylabel('Y position');
title('Random Forest'); legend; axis equal; hold off;

sgtitle(sprintf('Trial %d – Reconstructed Hand Trajectory', trialNum));
end

```

Normalize Trajectory Function

```

function trajNorm = normalizeTrajectory(traj)
% Normalize trajectory by path length for comparison

pathLength = sum(sqrt(sum(diff(traj, 1, 1).^2, 2)));
if pathLength > 0
    trajNorm = traj / pathLength;
else
    trajNorm = traj;
end
end

```

Actual vs Decoded velocity Function

```

function plotDecodedVelocity(decoderModel, R, trialNum)
% Plot actual vs decoded hand velocity over time for a given trial.
% Shows Vx and Vy over time for both ridge and random forest decoders,
% alongside the actual velocity.

binSizeMs = decoderModel.binSizeMs;
nUnits    = length(R(1).unit);

moveOnset = R(trialNum).moveOnsetTime;
moveEnds  = R(trialNum).moveEndsTime;
binEdges  = moveOnset : binSizeMs : moveEnds;
nBins     = length(binEdges) - 1;

% Binned spike times
trialNeural = zeros(nBins, nUnits);
for u = 1:nUnits
    spks = R(trialNum).unit(u).spikeTimes;
    trialNeural(:, u) = histcounts(spks, binEdges) / (binSizeMs / 1000);
end

trialNeural = sqrt(trialNeural);

```

```

% Predicted velocities
ridgeVel      = zeros(nBins, 2);
ridgeVel(:,1) = trialNeural * decoderModel.ridge.beta(2:end,1) +
decoderModel.ridge.beta(1,1);
ridgeVel(:,2) = trialNeural * decoderModel.ridge.beta(2:end,2) +
decoderModel.ridge.beta(1,2);

rfVel = [predict(decoderModel.rf.rfVx, trialNeural), ...
predict(decoderModel.rf.rfVy, trialNeural)];

% Actual velocity
handTimes = R(trialNum).HAND.times;
dt         = mean(diff(handTimes));
velX       = gradient(R(trialNum).HAND.X, dt);
velY       = gradient(R(trialNum).HAND.Y, dt);
binCenters = binEdges(1:end-1) + binSizeMs / 2;

actualVel    = zeros(nBins, 2);
actualVel(:,1) = interp1(handTimes, velX, binCenters);
actualVel(:,2) = interp1(handTimes, velY, binCenters);

% Time axis
timeMs = binCenters - moveOnset;

% R^2
r2Ridge = 1 - sum((actualVel(:) - ridgeVel(:)).^2) / sum((actualVel(:) -
mean(actualVel(:))).^2);
r2RF     = 1 - sum((actualVel(:) - rfVel(:)).^2) / sum((actualVel(:) -
mean(actualVel(:))).^2);

figure;

% Vx
subplot(2, 1, 1); hold on;
plot(timeMs, actualVel(:, 1), 'k-', 'LineWidth', 2, 'DisplayName', 'Actual');
plot(timeMs, ridgeVel(:, 1), 'b--', 'LineWidth', 1.5, ...
'DisplayName', sprintf('Ridge (R^2=%.2f)', r2Ridge));
plot(timeMs, rfVel(:, 1), 'r--', 'LineWidth', 1.5, ...
'DisplayName', sprintf('RF (R^2=%.2f)', r2RF));
xlabel('Time from movement onset (ms)');
ylabel('V_x (mm/s)');
title('X velocity');
legend('Location', 'best', 'FontSize', 8);
hold off;

% Vy
subplot(2, 1, 2); hold on;
plot(timeMs, actualVel(:, 2), 'k-', 'LineWidth', 2, 'DisplayName', 'Actual');
plot(timeMs, ridgeVel(:, 2), 'b--', 'LineWidth', 1.5, 'DisplayName', 'Ridge');

```

```

plot(timeMs, rfVel(:, 2), 'r--', 'LineWidth', 1.5, 'DisplayName', 'RF');
xlabel('Time from movement onset (ms)');
ylabel('V_y (mm/s)');
title('Y velocity');
legend('Location', 'best', 'FontSize', 8);
hold off;

sgtitle(sprintf('Trial %d – Actual vs Decoded Velocity', trialNum));
end

```

Classifier

```

% We will classify curved vs. straight reaches based on neural data from
% the delay and movement period separately using SVM and logistic
% regression to assess when kinematic information is encoded in M1.

% We will assess the significance of our classifier using a Monte Carlo
% shuffle test, where we shuffle curved vs straight trial labels 1000 times
% to generate a null distribution and compute a p-value for each
% method/time-window combination.

function results = classifier(R, varargin)

p = inputParser;
addParameter(p, 'BinSize', 50, @isnumeric);
addParameter(p, 'DelayWin', [-500 0], @(x) isnumeric(x) && numel(x)==2);
addParameter(p, 'MoveWin', [0 500], @(x) isnumeric(x) && numel(x)==2);
addParameter(p, 'nPerm', 1000, @isnumeric);
addParameter(p, 'nFolds', 5, @isnumeric);
addParameter(p, 'Methods', {'svm','logreg'},@iscell);
parse(p, varargin{:});
opt = p.Results;

bin_size = opt.BinSize;
delay_win = opt.DelayWin;
move_win = opt.MoveWin;
n_perm = opt.nPerm;
n_cv_folds = opt.nFolds;
methods = opt.Methods;
method_labels = cellfun(@(m) method_display_name(m), methods, ...
                        'UniformOutput', false);
n_methods = numel(methods);

window_labels = {'Delay Period', 'Movement Period'};
n_windows = numel(window_labels);

barrier_labels_str = {'Straight (No Barriers)', 'Curved (Barriers)'};
n_barrier = numel(barrier_labels_str);

```

```

angle_strs = {'Left', 'Lower-Left', 'Lower-Right', 'Upper-Right', 'Up', 'Upper-Left'};

conditionIDs = [R.conditionID];
straight_trials = find(mod(conditionIDs, 3) == 1);
curved_trials = find(mod(conditionIDs, 3) == 2);
all_trials = [straight_trials(:); curved_trials(:)];
n_trials = numel(all_trials);
n_units = numel(R(1).unit);

time_bins = delay_win(1) : bin_size : move_win(2);
bin_centers = time_bins(1:end-1) + bin_size/2;
n_bins = numel(bin_centers);

onset_align = [R.moveOnsetTime];
firing_rates = nan(n_trials, n_units, n_bins);

for t = 1:n_trials
    tr = all_trials(t);
    t0 = onset_align(tr);
    for u = 1:n_units
        spike_times = R(tr).unit(u).spikeTimes - t0;
        counts = histcounts(spike_times, time_bins);
        firing_rates(t, u, :) = counts / (bin_size / 1000);
    end
end
disp('Firing rates computed.');
```

```

delay_bins = bin_centers >= delay_win(1) & bin_centers < delay_win(2);
movement_bins = bin_centers >= move_win(1) & bin_centers < move_win(2);

targetXY = reshape([R.targetXY], 2, []);
angles = atan2d(targetXY(:,2), targetXY(:,1));
angleGroups = round(angles / 45) * 45;
angleGroups(angleGroups == 180) = -180;
angleGroups(angleGroups == 0 & angles < 5) = -45;
angleGroups(angleGroups == 0 & angles >= 5) = 45;
uniqueGroups = unique(angleGroups);
nGroups = length(uniqueGroups);

dir_labels_all = nan(numel(R), 1);
for g = 1:nGroups
    dir_labels_all(angleGroups == uniqueGroups(g)) = g;
end
chance_level_dir = 1 / nGroups;

straight_pos = 1:numel(straight_trials);
curved_pos = numel(straight_trials)+1 : n_trials;
barrier_conditions = {straight_pos, curved_pos};

for b = 1:n_barrier
```

```

    dir_labels = dir_labels_all(all_trials(barrier_conditions{b}));
    fprintf('\n%s:\n', barrier_labels_str{b});
    tabulate(dir_labels);
end

obs_acc = nan(n_methods, n_windows, n_barrier);
perm_acc = nan(n_methods, n_windows, n_barrier, n_perm);
p_values = nan(n_methods, n_windows, n_barrier);
pred_store = cell(n_methods, n_windows, n_barrier);
true_store = cell(n_methods, n_windows, n_barrier);

for b = 1:n_barrier
    trial_pos = barrier_conditions{b};
    trial_ids = all_trials(trial_pos);
    dir_labels = dir_labels_all(trial_ids);
    n_tr = numel(trial_pos);

    X_delay = squeeze(mean(firing_rates(trial_pos, :, delay_bins), 3));
    X_movement = squeeze(mean(firing_rates(trial_pos, :, movement_bins), 3));
    window_features = {X_delay, X_movement};
    cv = cvpartition(dir_labels, 'KFold', n_cv_folds, 'Stratify', true);

    for w = 1:n_windows
        X = window_features{w};
        for m = 1:n_methods
            fprintf('\n=====');
            fprintf('%s | %s | %s\n', method_labels{m}, ...
                window_labels{w}, barrier_labels_str{b});
            fprintf('=====');
            [pred, true_lab] = dd_classify_kfold_predictions( ...
                X, dir_labels, methods{m}, cv);
            pred_store{m,w,b} = pred;
            true_store{m,w,b} = true_lab;
            obs_acc(m,w,b) = mean(pred == true_lab);
            fprintf('Observed accuracy = %.3f\n', obs_acc(m,w,b));

            method_m = methods{m};
            perm_acc_tmp = nan(n_perm, 1);
            parfor pp = 1:n_perm
                shuf = dir_labels(randperm(n_tr));
                perm_acc_tmp(pp) = dd_classify_kfold( ...
                    X, shuf, method_m, cv);
            end
            perm_acc(m,w,b,:) = perm_acc_tmp;
            p_values(m,w,b) = ...
                (sum(perm_acc_tmp >= obs_acc(m,w,b)) + 1) / (n_perm + 1);
            fprintf('p-value = %.4f\n', p_values(m,w,b));
        end
    end
end
end

```

```

fprintf('\n%s\n', repmat('-',1,75));
fprintf('%-22s %-20s %-24s %-10s %-10s\n', ...
    'Method', 'Window', 'Condition', 'Accuracy', 'p-value');
fprintf('%s\n', repmat('-',1,75));
for b = 1:n_barrier
    for w = 1:n_windows
        for m = 1:n_methods
            pv = p_values(m,w,b);
            fprintf('%-22s %-20s %-24s %-10.3f %-10s %s\n', ...
                method_labels{m}, window_labels{w}, barrier_labels_str{b}, ...
                obs_acc(m,w,b), sprintf('%.4f',pv), dd_significance_label(pv));
        end
    end
end
fprintf('%s\n', repmat('-',1,75));

plot_confusion_matrices(pred_store, true_store, p_values, obs_acc, ...
    n_barrier, n_methods, n_windows, nGroups, ...
    barrier_labels_str, method_labels, window_labels, angle_strs);

results.obs_acc = obs_acc;
results.perm_acc = perm_acc;
results.p_values = p_values;
results.pred = pred_store;
results.true_lab = true_store;
results.params = opt;
end

function plot_confusion_matrices(pred_store, true_store, p_values, obs_acc, ...
    n_barrier, n_methods, n_windows, nGroups, ...
    barrier_labels_str, method_labels, window_labels, angle_strs)
for b = 1:n_barrier
    figure('Color','w', ...
        'Position',[100 100 1200 1000], ...
        'Name', barrier_labels_str{b});
    panel = 0;

    for m = 1:n_methods
        for w = 1:n_windows
            panel = panel + 1;
            ax = subplot(n_methods, n_windows, panel);
            pred = pred_store{m,w,b};
            true_lab = true_store{m,w,b};
            C = zeros(nGroups, nGroups);
            for i = 1:nGroups
                for j = 1:nGroups
                    C(i,j) = sum(true_lab == i & pred == j);
                end
            end
        end
    end
end

```

```

C_norm = C ./ sum(C, 2);

imagesc(ax, C_norm);
colormap(ax, 'sky');
clim(ax, [0 1]);
hold(ax, 'on');

for i = 1:nGroups
    for j = 1:nGroups
        val = C_norm(i,j);
        txt_col = 'k';
        if val > 0.6; txt_col = 'w'; end
        text(ax, j, i, sprintf('%.0f%%', val*100), ...
            'HorizontalAlignment','center', ...
            'VerticalAlignment','middle', ...
            'FontSize', 8, 'Color', txt_col);
    end
end

for i = 1:nGroups
    rectangle(ax, ...
        'Position', [i-0.5, i-0.5, 1, 1], ...
        'EdgeColor', [0.9 0.5 0.1], ...
        'LineWidth', 1.5);
end
pv = p_values(m,w,b);
if pv < 0.0001
    p_str = '< 0.0001';
else
    p_str = sprintf('%.4f', pv);
end
overall_acc = obs_acc(m,w,b) * 100;
ax.XTick = 1:nGroups;
ax.YTick = 1:nGroups;
ax.XTickLabel = angle_strs;
ax.YTickLabel = angle_strs;
ax.TickLength = [0 0];
ax.FontSize = 9;
ax.XTickLabelRotation = 45;
xlabel(ax, 'Predicted direction');
ylabel(ax, 'True direction');
title(ax, sprintf('%s | %s\nacc = %.1f%%, p = %s %s', ...
    method_labels{m}, window_labels{w}, ...
    overall_acc, p_str, dd_significance_label(pv)));
cb = colorbar(ax);
cb.Label.String = 'Proportion';
cb.FontSize = 8;
end
end
sgtitle(sprintf('Direction Confusion Matrix - %s', ...

```



```

        barrier_labels_str{b}), ...
        'FontSize', 13, 'FontWeight', 'bold');
    end
end

function acc = dd_classify_kfold(X, y, method, cv)
    correct = 0;
    for fold = 1:cv.NumTestSets
        [X_tr, X_te, y_tr, y_te] = split_fold(X, y, cv, fold);
        mdl = fit_model(X_tr, y_tr, method);
        pred = predict(mdl, X_te);
        correct = correct + sum(pred == y_te);
    end
    acc = correct / numel(y);
end

function [pred_all, true_all] = dd_classify_kfold_predictions(X, y, method, cv)
    pred_all = nan(size(y));
    true_all = y;
    for fold = 1:cv.NumTestSets
        test_idx = test(cv, fold);
        [X_tr, X_te, y_tr, ~] = split_fold(X, y, cv, fold);
        mdl = fit_model(X_tr, y_tr, method);
        pred_all(test_idx) = predict(mdl, X_te);
    end
end

function [X_train, X_test, y_train, y_test] = split_fold(X, y, cv, fold)
    train_idx = training(cv, fold);
    test_idx = test(cv, fold);
    X_train = X(train_idx, :);
    X_test = X(test_idx, :);
    y_train = y(train_idx);
    y_test = y(test_idx);
    mu = mean(X_train, 1);
    sigma = std(X_train, 0, 1);
    sigma(sigma==0) = 1;
    X_train = (X_train - mu) ./ sigma;
    X_test = (X_test - mu) ./ sigma;
end

function mdl = fit_model(X_train, y_train, method)
    switch method
        case 'svm'
            t = templateSVM('KernelFunction', 'rbf', ...
                'Standardize', false, ...
                'BoxConstraint', 1, ...
                'KernelScale', 'auto');
        case 'logreg'
            t = templateLinear('Learner', 'logistic', ...

```

```

                                'Regularization','ridge', ...
                                'Lambda', 1/size(X_train,1));

end
mdl = fitcecoc(X_train, y_train, 'Learners', t);
end

function label = dd_significance_label(p)
    if p < 0.001; label = '***';
    elseif p < 0.01; label = '**';
    elseif p < 0.05; label = '*';
    else; label = '(n.s.)';
    end
end

function name = method_display_name(method)
    switch method
        case 'svm'; name = 'SVM (RBF)';
        case 'logreg'; name = 'Logistic Regression';
        otherwise; name = method;
    end
end
end

```

Curvature Metric

% Behavioral: Find a curvature metric (ratio of actual path length to
 % straight line distance between start and end). Plot the distribution of
 % curvature with a bootstrapped confidence interval for curved and straight
 % path conditions. This would be a behavioral verification that the animal
 % actually did the curved paths and not just the straight length ones.

```

function curvatureMetric(R)
num_trials = numel(R);
conditionIDs = [R.conditionID];
straight_idx = mod(conditionIDs, 3) == 1;
curved_idx = mod(conditionIDs, 3) == 2;

moveOnsetTimes = [R.moveOnsetTime];
moveEndsTimes = [R.moveEndsTime];

curvature = nan(1, num_trials);

for trial = 1:num_trials
    hand_times = R(trial).HAND.times;
    x_pos = R(trial).HAND.X;
    y_pos = R(trial).HAND.Y;

    [~, onset_idx] = min(abs(hand_times - moveOnsetTimes(trial)));
    [~, end_idx] = min(abs(hand_times - moveEndsTimes(trial)));

```

```

x_seg = x_pos(onset_idx:end_idx);
y_seg = y_pos(onset_idx:end_idx);

coords      = [x_seg(:), y_seg(:)];
segment_lengths = sqrt(sum(diff(coords).^2, 2));
total_path_length = sum(segment_lengths);
straight_line_dist = sqrt((x_seg(end) - x_seg(1))^2 + ...
    (y_seg(end) - y_seg(1))^2);

curvature(trial) = total_path_length / straight_line_dist;
end
straight_curv = curvature(straight_idx & ~isnan(curvature));
curved_curv   = curvature(curved_idx   & ~isnan(curvature));
figure;

subplot(1, 2, 1);
histogram(straight_curv, 30, 'FaceColor', [0 0 0.8], ...
    'EdgeColor', 'none', 'Normalization', 'probability');
xlabel('Curvature Ratio');
ylabel('Proportion of Trials');
title('No Barriers');
xlim([0.5 3]);

subplot(1, 2, 2);
histogram(curved_curv, 30, 'FaceColor', 'red', ...
    'EdgeColor', 'none', 'Normalization', 'probability');
xlabel('Curvature Ratio');
ylabel('Proportion of Trials');
title('Barriers');
xlim([0.5 3]);

sgtitle('Path Curvature Distribution by Condition');
n_bootstrap = 10000;

straight_random_inds = randi(numel(straight_curv), numel(straight_curv),
n_bootstrap);
straight_bootstrap   = mean(straight_curv(straight_random_inds), 1);

curved_random_inds = randi(numel(curved_curv), numel(curved_curv), n_bootstrap);
curved_bootstrap   = mean(curved_curv(curved_random_inds), 1);

straight_mean = mean(straight_curv);
straight_ci   = prctile(straight_bootstrap, [2.5, 97.5]);

curved_mean = mean(curved_curv);
curved_ci   = prctile(curved_bootstrap, [2.5, 97.5]);
figure;
hold on;

histogram(straight_bootstrap, 50, 'FaceColor', [0 0 0.8], 'FaceAlpha', 0.5, ...

```

```

    'EdgeColor', 'none', 'Normalization', 'probability');
histogram(curved_bootstrap, 50, 'FaceColor', 'red', 'FaceAlpha', 0.5, ...
    'EdgeColor', 'none', 'Normalization', 'probability');

yl = ylim;

fill([straight_ci(1) straight_ci(2) straight_ci(2) straight_ci(1)], ...
    [0 0 yl(2) yl(2)], [0 0 0.8], 'FaceAlpha', 0.15, 'EdgeColor', 'none');
fill([curved_ci(1) curved_ci(2) curved_ci(2) curved_ci(1)], ...
    [0 0 yl(2) yl(2)], 'red', 'FaceAlpha', 0.15, 'EdgeColor', 'none');

xline(straight_mean, 'b-', 'LineWidth', 2);
xline(curved_mean, 'r-', 'LineWidth', 2);

legend('No Barriers bootstrap', 'Barriers bootstrap', ...
    'No Barriers 95% CI', 'Barriers 95% CI', ...
    'No Barriers mean', 'Barriers mean', ...
    'Location', 'best');
xlabel('Mean Curvature Ratio');
ylabel('Proportion of Bootstrap Samples');
title('Bootstrap Mean Distributions with 95% CIs');

end

```

Hand Velocity, Monte Carlo

```

% Behavioral: Hand velocity over time split by the curved and straight
% reaches. Compute peak hand speed during trial, split trials into groups
% based on curved or straight reaches, compare the means. Monte Carlo:
% shuffle the curved/straight labels, recompute the difference in the
% means, do this 1,000 times to create a null distribution. P-value would
% show us if curved and straight reaches have significantly different peak
% hand speeds.

```

```

%% Find peak speeds
function handVelocityMonteCarlo(R)
num_trials = numel(R);
conditionIDs = [R.conditionID];
straight_idx = mod(conditionIDs, 3) == 1;
curved_idx = mod(conditionIDs, 3) == 2;

moveOnsetTimes = [R.moveOnsetTime];
moveEndsTimes = [R.moveEndsTime];

peak_speeds = nan(1, num_trials);

for trial = 1:num_trials
    hand_times = R(trial).HAND.times(:);
    x_pos = R(trial).HAND.X(:);
    y_pos = R(trial).HAND.Y(:);

```

```

[~, onset_idx] = min(abs(hand_times - moveOnsetTimes(trial)));
[~, end_idx] = min(abs(hand_times - moveEndsTimes(trial)));

x_seg = x_pos(onset_idx:end_idx);
y_seg = y_pos(onset_idx:end_idx);
dt = diff(hand_times(onset_idx:end_idx));
dx = diff(x_seg);
dy = diff(y_seg);

valid = dt > 0;
dt = dt(valid);
dx = dx(valid);
dy = dy(valid);

speed = sqrt((dx ./ dt).^2 + (dy ./ dt).^2);
peak_speeds(trial) = max(speed);
end

straight_speed = peak_speeds(straight_idx & ~isnan(peak_speeds));
curved_speed = peak_speeds(curved_idx & ~isnan(peak_speeds));
observed_diff = mean(curved_speed) - mean(straight_speed);

figure;
subplot(1, 2, 1);
histogram(straight_speed, 30, 'FaceColor', [0 0 0.8], ...
    'EdgeColor', 'none', 'Normalization', 'probability');
hold on;
xline(mean(straight_speed), 'b--', 'LineWidth', 2);
xlabel('Peak Hand Speed');
ylabel('Proportion of Trials');
title('No Barriers');
subplot(1, 2, 2);
histogram(curved_speed, 30, 'FaceColor', 'red', ...
    'EdgeColor', 'none', 'Normalization', 'probability');
hold on;
xline(mean(curved_speed), 'r--', 'LineWidth', 2);
xlabel('Peak Hand Speed');
ylabel('Proportion of Trials');
title('Barriers');
sgtitle('Peak Hand Speed Distribution by Condition');

n_perm = 10000;
permutation_diffs = nan(1, n_perm);
pooled_speeds = [straight_speed, curved_speed];
num_straight = numel(straight_speed);

for i = 1:n_perm
    shuffled = pooled_speeds(randperm(numel(pooled_speeds)));
    perm_straight = shuffled(1:num_straight);

```

```

    perm_curved = shuffled(num_straight+1:end);
    permutation_diffs(i) = mean(perm_curved) - mean(perm_straight);
end

p_value = mean(abs(permutation_diffs) >= abs(observed_diff));

figure;
histogram(permutation_diffs, 50, 'FaceColor', [0.5 0.5 0.5], ...
    'EdgeColor', 'none', 'Normalization', 'probability');
hold on;
xline( observed_diff, 'r--', 'LineWidth', 2);
xline(-observed_diff, 'r--', 'LineWidth', 2);

xlabel('Difference in Mean Peak Speed (Curved - Straight)');
ylabel('Proportion of Permutations');
title(sprintf('Null Distribution (Monte Carlo Permutation Test)\nObserved
difference = %.3f, p = %.3f', ...
    observed_diff, p_value));
legend('Null distribution', 'Observed difference');

end

```

Linear Dimensionality Reduction

```

% We will do cross-validated PCA on M1 activity to determine the true low
% dimensional structure of the neuron's responses, comparing dimensionality
% under no normalization, z-scoring, and soft normalization.

```

```

function results = linearDimensionalityReduction(R, varargin)
p = inputParser;
addParameter(p, 'PreWindow', 150, @isnumeric);
addParameter(p, 'PostWindow', 50, @isnumeric);
addParameter(p, 'BinSize', 20, @isnumeric);
addParameter(p, 'nFolds', 5, @isnumeric);
addParameter(p, 'SoftNorm', 5, @isnumeric);
parse(p, varargin{:});
opt = p.Results;

pre_window = opt.PreWindow;
post_window = opt.PostWindow;
bin_size = opt.BinSize;
k = opt.nFolds;
soft_norm = opt.SoftNorm;

conditionIDs = [R.conditionID];
straight_trials = find(mod(conditionIDs, 3) == 1);
curved_trials = find(mod(conditionIDs, 3) == 2);
all_trials = [straight_trials(:); curved_trials(:)];
n_trials = numel(all_trials);

```

```

n_units = numel(R(1).unit);
n_straight = numel(straight_trials);
n_curved = numel(curved_trials);

time_bins = -pre_window : bin_size : post_window;
bin_centers = time_bins(1:end-1) + bin_size/2;
n_bins = numel(bin_centers);

onset_align = [R.moveOnsetTime];
firing_rates = nan(n_trials, n_units, n_bins);

for t = 1:n_trials
    tr = all_trials(t);
    t0 = onset_align(tr);
    for u = 1:n_units
        spike_times = R(tr).unit(u).spikeTimes - t0;
        counts = histcounts(spike_times, time_bins);
        firing_rates(t, u, :) = counts / (bin_size / 1000);
    end
end
disp('Firing rates created.');
```

```

fr_straight = squeeze(mean(firing_rates(1:n_straight, :, :), 1));
fr_curved = squeeze(mean(firing_rates(n_straight+1:end, :, :), 1));
fr_cond_avg = permute(cat(3, fr_straight, fr_curved), [3 1 2]);
n_conds = 2;

cond_labels = [ones(n_straight, 1); 2*ones(n_curved, 1)];
fold_ids_trials = zeros(n_trials, 1);
for cond = 1:2
    cond_mask = find(cond_labels == cond);
    n_cond = numel(cond_mask);
    perm = randperm(n_cond);
    cond_folds = mod(0:n_cond-1, k) + 1;
    fold_ids_trials(cond_mask) = cond_folds(perm);
end

methods = {'none', 'zscore', 'soft'};
normalization_labels = {'No Normalization', 'Z-Score', 'Soft Normalization'};
colors_method = [0 0 0; 0 0 1; 1 0 0];
n_methods = numel(methods);

n_X_fixed = floor(n_units / 2);
max_dims = n_X_fixed - 1;
n_plot_dims = min(n_X_fixed - 1, 2*n_bins - 1);

cv_loss = nan(n_methods, k, k, max_dims);
all_explained = nan(n_methods, n_plot_dims);
all_score = cell(n_methods, 1);
all_coeff = cell(n_methods, 1);

```

```

for m = 1:n_methods
    for fold_r = 1:k
        train_idx = find(fold_ids_trials ~= fold_r);
        test_idx = find(fold_ids_trials == fold_r);
        n_train = numel(train_idx);
        n_test = numel(test_idx);

        for fold_c = 1:k
            neuron_perm = randperm(n_units);
            x_neurons = neuron_perm(1:n_X_fixed);
            y_neurons = neuron_perm(n_X_fixed+1:end);
            n_Y = numel(y_neurons);

            X_train = reshape(permute(firing_rates(train_idx, x_neurons, :), [1 3 2]), n_train*n_bins, n_X_fixed);
            Y_train = reshape(permute(firing_rates(train_idx, y_neurons, :), [1 3 2]), n_train*n_bins, n_Y);
            X_test = reshape(permute(firing_rates(test_idx, x_neurons, :), [1 3 2]), n_test*n_bins, n_X_fixed);
            Y_test = reshape(permute(firing_rates(test_idx, y_neurons, :), [1 3 2]), n_test*n_bins, n_Y);

            [X_train_n, X_test_n] = ldr_normalize(X_train, X_test, methods{m}, soft_norm);

            mu_X = mean(X_train_n, 1);
            X_train_c = X_train_n - mu_X;
            X_test_c = X_test_n - mu_X;
            mu_Y = mean(Y_train, 1);
            Y_train_c = Y_train - mu_Y;

            [coeff, score, ~] = pca(X_train_c, 'NumComponents', max_dims);

            for d = 1:max_dims
                S_train = score(:, 1:d);
                beta = S_train \ Y_train_c;
                S_test = X_test_c * coeff(:, 1:d);
                Y_hat = S_test * beta + mu_Y;
                cv_loss(m, fold_r, fold_c, d) = mean((Y_test - Y_hat).^2, 'all');
            end
        end
    end

    FR_all = reshape(permute(firing_rates, [1 3 2]), n_trials*n_bins, n_units);
    FR_all_tmp = FR_all;
    [FR_all_n, ~] = ldr_normalize(FR_all_tmp, FR_all_tmp, methods{m}, soft_norm);

    FR_all_n = reshape(FR_all_n, n_trials, n_bins, n_units);
    psth_s = squeeze(mean(FR_all_n(1:n_straight, :, :), 1));

```



```

psth_c = squeeze(mean(FR_all_n(n_straight+1:end, :, :), 1));

X_plot_final = [psth_s; psth_c];
X_plot_final = X_plot_final - mean(X_plot_final, 1);

[coeff_full, score_full, ~, ~, explained] = pca(X_plot_final);
all_explained(m, :) = explained(1:n_plot_dims)';
all_score{m} = score_full;
all_coeff{m} = coeff_full;
end

mean_loss = squeeze(mean(cv_loss, [2 3]));

ldr_plot_cv_loss(mean_loss, max_dims, n_methods, normalization_labels,
colors_method);
ldr_plot_scree(all_explained, n_plot_dims, n_methods, normalization_labels,
colors_method);
ldr_plot_cumulative(all_explained, n_plot_dims, n_methods, normalization_labels,
colors_method);

results.cv_loss = cv_loss;
results.mean_loss = mean_loss;
results.explained = all_explained;
results.score = all_score;
results.coeff = all_coeff;
results.bin_centers = bin_centers;
results.params = opt;
end

function [X_n, X_test_n] = ldr_normalize(X_train, X_test, method, soft_norm)
    switch method
        case 'none'
            X_n = X_train;
            X_test_n = X_test;
        case 'zscore'
            mu = mean(X_train, 1);
            sig = std(X_train, 0, 1);
            sig(sig == 0) = 1;
            X_n = (X_train - mu) ./ sig;
            X_test_n = (X_test - mu) ./ sig;
        case 'soft'
            rng = max(X_train, [], 1) - min(X_train, [], 1);
            X_n = X_train ./ (rng + soft_norm);
            X_test_n = X_test ./ (rng + soft_norm);
    end
end

function ldr_plot_cv_loss(mean_loss, max_dims, n_methods, norm_labels, colors)
    figure('Color', 'w');
    for m = 1:n_methods

```

```

        subplot(1, n_methods, m);
        plot(1:max_dims, mean_loss(m,:), '-o', ...
            'Color', colors(m,:), 'LineWidth', 2);
        xlabel('Number of PCs (k)');
        ylabel('MSE');
        title(norm_labels{m});
        grid on;
    end
    sgtitle('Cross-validated PCA loss by dimensionality');
end

function ldr_plot_scree(all_explained, n_plot_dims, n_methods, norm_labels, colors)
    n_plot = min(20, n_plot_dims);
    figure('Color', 'w'); hold on;
    for m = 1:n_methods
        plot(1:n_plot, all_explained(m, 1:n_plot), '-o', ...
            'Color', colors(m,:), 'LineWidth', 2);
    end
    xlabel('Principal component');
    ylabel('Variance explained (%)');
    title('Scree plot');
    legend(norm_labels, 'Location', 'northeast');
    grid on;
end

function ldr_plot_cumulative(all_explained, n_plot_dims, n_methods, norm_labels,
    colors)
    n_plot = min(20, n_plot_dims);
    figure('Color', 'w'); hold on;
    for m = 1:n_methods
        cum_exp = cumsum(all_explained(m, :));
        plot(1:n_plot, cum_exp(1:n_plot), '-o', ...
            'Color', colors(m,:), 'LineWidth', 2);
    end
    yline(80, 'k--', 'LineWidth', 1.5, 'Label', '80%');
    yline(95, 'k:', 'LineWidth', 1.5, 'Label', '95%');
    xlabel('Number of components');
    ylabel('Cumulative variance explained (%)');
    title('Cumulative variance');
    ylim([0 100]);
    legend(norm_labels, 'Location', 'southeast');
    grid on;
end

```

PCA Trajectories

```

function pcaTrajectories(R)

pre_window = 500;

```

```

post_window = 1500;
bin_size = 20;
k = 5;
soft_norm = 5;
soft_norm_alpha = 5;

methods = {'none', 'zscore', 'soft'};
normalization_labels = {'No Normalization', 'Z-Score', 'Soft Normalization'};

conditionIDs = [R.conditionID];
straight_idx = mod(conditionIDs, 3) == 1;
curved_idx = mod(conditionIDs, 3) == 2;
straight_trials = find(straight_idx);
curved_trials = find(curved_idx);

time_bins = -pre_window : bin_size : post_window;
bin_centers = time_bins(1:end-1) + bin_size/2;
n_bins = numel(bin_centers);
n_units = numel(R(1).unit);

onset_align = [R.moveOnsetTime];
all_trials = [straight_trials(:); curved_trials(:)];
n_trials = numel(all_trials);

firing_rates = nan(n_trials, n_units, n_bins);
for t = 1:n_trials
    tr = all_trials(t);
    t0 = onset_align(tr);
    for u = 1:n_units
        spike_times = R(tr).unit(u).spikeTimes - t0;
        counts = histcounts(spike_times, time_bins);
        firing_rates(t, u, :) = counts / (bin_size / 1000);
    end
end
disp('Firing rates created.')

fr_straight = squeeze(mean(firing_rates(1:numel(straight_trials), :, :), 1));
fr_curved = squeeze(mean(firing_rates(numel(straight_trials)+1:end, :, :), 1));
fr_cond_avg = permute(cat(3, fr_straight, fr_curved), [3 1 2]);
n_conds = 2;

n_straight = numel(straight_trials);
n_curved = numel(curved_trials);
cond_labels = [ones(n_straight, 1); 2*ones(n_curved, 1)];

fold_ids_trials = zeros(n_trials, 1);
for cond = 1:2
    cond_mask = find(cond_labels == cond);
    n_cond = numel(cond_mask);
    perm = randperm(n_cond);

```

```

    cond_folds = mod(0:n_cond-1, k) + 1;
    fold_ids_trials(cond_mask) = cond_folds(perm);
end

n_X_fixed = floor(n_units / 2);
max_dims = n_X_fixed - 1;
n_plot_dims = min(n_X_fixed - 1, 2*n_bins - 1);

cv_loss = nan(numel(methods), k, k, max_dims);
all_explained = nan(numel(methods), n_plot_dims);
all_score = cell(numel(methods), 1);
all_coeff = cell(numel(methods), 1);

for m = 1:numel(methods)
    for fold_r = 1:k
        train_idx = find(fold_ids_trials ~= fold_r);
        test_idx = find(fold_ids_trials == fold_r);
        n_train = numel(train_idx);
        n_test = numel(test_idx);

        for fold_c = 1:k
            neuron_perm = randperm(n_units);
            x_neurons = neuron_perm(1:n_X_fixed);
            y_neurons = neuron_perm(n_X_fixed+1:end);
            n_Y = numel(y_neurons);

            X_train = reshape(permute(firing_rates(train_idx, x_neurons, :), [1 3
2])), n_train*n_bins, n_X_fixed);
            Y_train = reshape(permute(firing_rates(train_idx, y_neurons, :), [1 3
2])), n_train*n_bins, n_Y);
            X_test = reshape(permute(firing_rates(test_idx, x_neurons, :), [1 3
2])), n_test*n_bins, n_X_fixed);
            Y_test = reshape(permute(firing_rates(test_idx, y_neurons, :), [1 3
2])), n_test*n_bins, n_Y);

            switch methods{m}
                case 'none'
                    X_train_n = X_train; X_test_n = X_test;
                case 'zscore'
                    mu_x = mean(X_train, 1);
                    sig_x = std(X_train, 0, 1); sig_x(sig_x == 0) = 1;
                    X_train_n = (X_train - mu_x) ./ sig_x;
                    X_test_n = (X_test - mu_x) ./ sig_x;
                case 'soft'
                    rng_x = max(X_train, [], 1) - min(X_train, [], 1);
                    X_train_n = X_train ./ (rng_x + soft_norm);
                    X_test_n = X_test ./ (rng_x + soft_norm);
            end

            mu_X = mean(X_train_n, 1);
            X_train_c = X_train_n - mu_X;

```

```

X_test_c = X_test_n - mu_X;
mu_Y = mean(Y_train, 1);
Y_train_c = Y_train - mu_Y;
[coeff, score, ~] = pca(X_train_c, 'NumComponents', max_dims);
for d = 1:max_dims
    S_train = score(:, 1:d);
    beta = S_train \ Y_train_c;
    S_test = X_test_c * coeff(:, 1:d);
    Y_hat = S_test * beta + mu_Y;
    cv_loss(m, fold_r, fold_c, d) = mean((Y_test - Y_hat).^2, 'all');
end
end
end

FR_all = reshape(permute(firing_rates, [1 3 2]), n_trials*n_bins, n_units);
switch methods{m}
case 'none'
    FR_all_n = FR_all;
case 'zscore'
    mu_x = mean(FR_all, 1);
    sig_x = std(FR_all, 0, 1); sig_x(sig_x == 0) = 1;
    FR_all_n = (FR_all - mu_x) ./ sig_x;
case 'soft'
    rng_x = max(FR_all, [], 1) - min(FR_all, [], 1);
    FR_all_n = FR_all ./ (rng_x + soft_norm);
end

FR_all_n = reshape(FR_all_n, n_trials, n_bins, n_units);
psth_s = squeeze(mean(FR_all_n(1:n_straight, :, :), 1));
psth_c = squeeze(mean(FR_all_n(n_straight+1:end, :, :), 1));
X_plot_final = [psth_s; psth_c];
X_plot_final = X_plot_final - mean(X_plot_final, 1);

[coeff_full, score_full, ~, ~, explained] = pca(X_plot_final);
all_score{m} = score_full;
all_explained(m, :) = explained(1:n_plot_dims)';
all_coeff{m} = coeff_full;
end

mean_loss = squeeze(mean(cv_loss, [2 3]));
figure('Position', [100 100 1300 500]);
pc_colors = nebula(3);

for m = 1:numel(methods)
    subplot(1, 3, m); hold on;
    score = all_score{m};
    for pc = 1:3
        plot(bin_centers, score(1:n_bins, pc), '-', 'Color', pc_colors(pc,:),
'LineWidth', 2);

```

```

        plot(bin_centers, score(n_bins+1:end, pc), '--', 'Color', pc_colors(pc,:),
'LineWidth', 2);
    end
    xline(0, 'k--', 'LineWidth', 1.5);
    xlabel('Time from movement onset (ms)');
    ylabel('PC score');
    title(normalization_labels{m});
    if m == 1
        legend('PC1 straight', 'PC1 curved', ...
            'PC2 straight', 'PC2 curved', ...
            'PC3 straight', 'PC3 curved', ...
            'Location', 'best');
    end
end
sgtitle('Top 3 PCs Over Time');
targetXY = reshape([R.targetXY], 2, []);
angles = atan2d(targetXY(:,2), targetXY(:,1));
angleGroups = round(angles / 45) * 45;
angleGroups(angleGroups == 180) = -180;
angleGroups(angleGroups == 0 & angles < 5) = -45;
angleGroups(angleGroups == 0 & angles >= 5) = 45;
uniqueGroups = unique(angleGroups);
nGroups = length(uniqueGroups);
dirLabels = {'Left', 'Lower-Left', 'Lower-Right', 'Upper-Right', 'Up', 'Upper-Left'};

all_trial_ids = 1:numel(R);
n_all = numel(all_trial_ids);

fr_all = nan(n_all, n_units, n_bins);
for t = 1:n_all
    tr = all_trial_ids(t);
    t0 = onset_align(tr);
    for u = 1:n_units
        spike_times = R(tr).unit(u).spikeTimes - t0;
        counts = histcounts(spike_times, time_bins);
        fr_all(t, u, :) = counts / (bin_size / 1000);
    end
end

psth_groups = nan(n_units, nGroups * n_bins);
for g = 1:nGroups
    group_trials = find(angleGroups == uniqueGroups(g));
    psth_groups(:, (g-1)*n_bins + (1:n_bins)) = ...
        squeeze(mean(fr_all(group_trials, :, :), 1));
end

fr_range = max(psth_groups, [], 2) - min(psth_groups, [], 2);
X_dir = psth_groups ./ (fr_range + soft_norm_alpha);
X_dir = X_dir - mean(X_dir, 2);
[~, score_dir, ~, ~, explained_dir] = pca(X_dir');

```

```

dir_colors = hsv(nGroups);
[~, onset_bin] = min(abs(bin_centers));
figure('Name', '3D Neural Trajectories', 'Position', [100 100 900 700]);
hold on;
h_legend = gobjects(nGroups, 1);
for g = 1:nGroups
    idx = (g-1)*n_bins + (1:n_bins);
    pc1 = score_dir(idx, 1);
    pc2 = score_dir(idx, 2);
    pc3 = score_dir(idx, 3);
    col = dir_colors(g, :);
    n_seg = n_bins - 1;
    alphas = linspace(0.25, 1.0, n_seg);
    for s = 1:n_seg
        seg_col = col .* alphas(s) + (1 - alphas(s)) * [1 1 1];
        seg_col = min(max(seg_col, 0), 1);
        plot3(pc1(s:s+1), pc2(s:s+1), pc3(s:s+1), '-', ...
            'Color', seg_col, 'LineWidth', 2.5, 'HandleVisibility', 'off');
    end

    h_legend(g) = plot3(nan, nan, nan, '-', ...
        'Color', col, 'LineWidth', 2, ...
        'DisplayName', sprintf('%d°', uniqueGroups(g)));
    plot3(pc1(1), pc2(1), pc3(1), '^', ...
        'MarkerFaceColor', col, 'MarkerEdgeColor', 'k', ...
        'MarkerSize', 7, 'HandleVisibility', 'off');
    plot3(pc1(onset_bin), pc2(onset_bin), pc3(onset_bin), 'o', ...
        'MarkerFaceColor', col, 'MarkerEdgeColor', 'k', ...
        'MarkerSize', 8, 'HandleVisibility', 'off');
end

plot3(0, 0, 0, 'k+', 'MarkerSize', 12, 'LineWidth', 2, 'DisplayName', 'Origin');
xlabel('PC1'); ylabel('PC2'); zlabel('PC3');
title('3D neural trajectories by reach direction (soft norm, all-data PCA)');
legend(dirLabels, 'Location', 'best', 'NumColumns', 2);
annotation('textbox', [0.01 0.01 0.3 0.06], ...
    'String', '▲ = start    ● = movement onset', ...
    'EdgeColor', 'none', 'FontSize', 9, 'Color', [0.4 0.4 0.4]);
grid on; axis equal; view(35, 25); box off;
set(gca, 'FontSize', 12);

end

```

Trial to Trial Variability

```

% We will compute the Fano Factor as a function of time aligned to both the
% target appearance, movement onset, and go-cue separately for curved and straight
% reaches to determine how variability evolves across delay and movement
% and whether this differs for the reach types, considering curved reaches

```

% are arguably more complex.

```
function trialToTrialVariability(R)
conditionIDs = [R.conditionID];
straight_idx = mod(conditionIDs, 3) == 1;
curved_idx = mod(conditionIDs, 3) == 2;

straight_trials = find(straight_idx);
curved_trials = find(curved_idx);

bin_size = 50;
step_size = 10;
pre_window = 500;
post_window = 800;

bin_centers = (-pre_window + bin_size/2) : step_size : (post_window - bin_size/2);
num_bins = numel(bin_centers);
num_units = numel(R(1).unit);
target_align = [R.targetAppearsTime];
gocue_align = [R.goCueTime];
onset_align = [R.moveOnsetTime];

straight_dir_ids = floor(conditionIDs(straight_idx) / 3);
curved_dir_ids = floor(conditionIDs(curved_idx) / 3);

unique_straight_dirs = unique(straight_dir_ids);
unique_curved_dirs = unique(curved_dir_ids);

ff_target = nan(num_units, num_bins, 2);
ff_gocue = nan(num_units, num_bins, 2);
ff_onset = nan(num_units, num_bins, 2);
for unit = 1:num_units
    for cond = 1:2
        if cond == 1
            trial_ids = straight_trials;
            dir_ids = straight_dir_ids;
            unique_dirs = unique_straight_dirs;
        else
            trial_ids = curved_trials;
            dir_ids = curved_dir_ids;
            unique_dirs = unique_curved_dirs;
        end

        num_dirs = numel(unique_dirs);

        ff_target_by_dir = nan(num_dirs, num_bins);
        ff_gocue_by_dir = nan(num_dirs, num_bins);
        ff_onset_by_dir = nan(num_dirs, num_bins);
    end
end
```



```

for d = 1:num_dirs
    dir_mask = dir_ids == unique_dirs(d);
    dir_trial_ids = trial_ids(dir_mask);
    num_dir_trials = numel(dir_trial_ids);

    counts_target = nan(num_dir_trials, num_bins);
    counts_gocue = nan(num_dir_trials, num_bins);
    counts_onset = nan(num_dir_trials, num_bins);

    for trial = 1:num_dir_trials
        tr = dir_trial_ids(trial);
        spike_times = R(tr).unit(unit).spikeTimes;

        rel_target = spike_times - target_align(tr);
        rel_gocue = spike_times - gocue_align(tr);
        rel_onset = spike_times - onset_align(tr);

        for b = 1:num_bins
            t_lo = bin_centers(b) - bin_size/2;
            t_hi = bin_centers(b) + bin_size/2;

            counts_target(trial, b) = sum(rel_target >= t_lo & rel_target <
t_hi);
            counts_gocue( trial, b) = sum(rel_gocue  >= t_lo & rel_gocue  <
t_hi);
            counts_onset( trial, b) = sum(rel_onset  >= t_lo & rel_onset  <
t_hi);
        end
    end

    ff_target_by_dir(d, :) = compute_fano_factor(counts_target);
    ff_gocue_by_dir( d, :) = compute_fano_factor(counts_gocue);
    ff_onset_by_dir( d, :) = compute_fano_factor(counts_onset);
end

ff_target(unit, :, cond) = nanmean(ff_target_by_dir, 1);
ff_gocue( unit, :, cond) = nanmean(ff_gocue_by_dir, 1);
ff_onset( unit, :, cond) = nanmean(ff_onset_by_dir, 1);
end

end

mean_ff = @(ff, cond) nanmean(ff(:, :, cond), 1);
sem_ff  = @(ff, cond) nanstd(ff(:, :, cond), 0, 1) /
sqrt(sum(~all(isnan(ff(:, :, cond)), 2)));

colors = struct('straight', [0 0 0.8], 'curved', [0.8 0 0]);
align_labels = {'Target Appearance', 'Go Cue', 'Movement Onset'};
x_labels = {'Time from Target Appearance (ms)', ...
            'Time from Go Cue (ms)', ...
            'Time from Movement Onset (ms)'};

```

```

ff_all = {ff_target, ff_gocue, ff_onset};

figure('Position', [100 100 1000 500]);
for a = 1:3
    subplot(1, 3, a);
    hold on;
    ff = ff_all{a};
    for cond = 1:2
        m = mean_ff(ff, cond);
        sem = sem_ff(ff, cond);

        if cond == 1
            col = colors.straight;
            lbl = 'Straight';
            fill_lbl = 'SEM Straight';
        else
            col = colors.curved;
            lbl = 'Curved';
            fill_lbl = 'SEM Curved';
        end

        fill([bin_centers, fliplr(bin_centers)], ...
            [m + sem, fliplr(m - sem)], ...
            col, 'FaceAlpha', 0.2, 'EdgeColor', 'none', ...
            'DisplayName', fill_lbl);
        plot(bin_centers, m, '-', 'Color', col, 'LineWidth', 2, ...
            'DisplayName', lbl);
    end

    xline(0, 'k--', 'LineWidth', 1.5, 'HandleVisibility', 'off');
    yline(1, 'k:', 'LineWidth', 1.0, 'HandleVisibility', 'off');

    xlabel(x_labels{a});
    ylabel('Fano Factor');
    title(aligned_labels{a});
    xlim([-pre_window, post_window]);

    if a == 1
        legend('Location', 'best');
    end
end
sgtitle('Population Fano Factor');
end

function ff = compute_fano_factor(counts)
    m = mean(counts, 1);
    v = var(counts, 0, 1);
    ff = nan(size(m));
    valid = m > 0;
    ff(valid) = v(valid) ./ m(valid);
end

```

end